

TRABALLO FIN DE GRAO
GRAO EN CIENCIA E ENXEÑARÍA DE DATOS

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante	Héctor Aldao Amoedo
Dirección:	Alejeandro Rodriguez Árias Samuel Magaz Romero

A Coruña, junio de 2026.

Dedicatoria¿?

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. ¿?

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Palabras clave:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Keywords:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.1.1	Definición del Problema	1
1.1.2	Objetivos	2
1.1.3	Alcance y Limitaciones	2
1.1.4	Metodología de Trabajo	2
1.1.5	Estructura de la Memoria	2
2	Estado del Arte	3
3	Herramientas y técnicas	7
3.1	Godot como motor de desarrollo	7
3.2	Control de versiones y despliegue	10
4	Metodología y planificación	12
5	Marco teórico	13
5.1	Árboles de decisión e ID3	13
5.2	Redes neuronales multicapa	15
6	Desarrollo	18
6.1	Prototipo: versión 0.1	18
6.1.1	Primeras Escenas	19
6.1.2	Exportación web con GitHub Pages	20
6.2	Árbol de decisión: versión 0.2	20
6.2.1	Primera arquitectura del árbol	21
6.2.2	Reorganización de la Escena y primeros elementos explicativos	23
6.2.3	Animación de la construcción del árbol	25
6.2.4	Gráficas y casos explicativos en la ventana	27

6.2.5	Primer panel de selección de datos	29
6.2.6	Carga de archivos y navegación por pasos	31
6.2.7	Primera evaluación visual del árbol	32
6.3	Red neuronal: versión 0.3	34
6.3.1	Primera arquitectura visual de la red	34
6.3.2	Configuración de la arquitectura y representación lógica	36
6.3.3	Selección de datos y restricciones de la red	38
6.3.4	Algoritmo de entrenamiento incremental	41
6.3.5	Forward pass y visualización de datos	42
6.3.6	Cálculo del error y backward pass	44
6.3.7	Ventana informativa de la red neuronal	45
6.3.8	Inferencia con la red entrenada	46
6.4	Mejoras de funcionalidades e interfaz de usuario	47
6.4.1	Entrada principal y navegación común	47
6.4.2	Adaptación a web, móvil y pantallas de distinto tamaño	50
6.4.3	Identidad visual y legibilidad de la interfaz	52
6.4.4	Renderizado de fórmulas matemáticas	54
6.4.5	Selección de datasets y validaciones de inferencia	54
6.4.6	Evaluación visual del árbol de decisión	55
6.4.7	Gráficas de activación e inspección de neuronas y nodos	57
6.4.8	Persistencia, carga y validación de modelos	58
7	Conclusiones	60
7.1	Resultados obtenidos y validación	60
8	Trabajo futuro	65
8.1	Ampliación del catálogo de modelos	65
8.2	Profundización en los modelos existentes	66
8.3	Integración con simulaciones interactivas	67
A	Material adicional	70
	Bibliografía	73

Índice de figuras

2.1	Sección interactiva de la página web de 3Blue1Brown.	5
2.2	Página web TensorFlow Playground.	5
3.1	Entorno de desarrollo de Godot.	8
3.2	Diagrama general de la arquitectura de Godot.	9
6.1	Versión preliminar del menú principal.	19
6.2	Primera Escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda se ve una sencilla estructura con forma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.	20
6.3	Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.	22
6.4	Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.	23
6.5	Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.	24
6.6	Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.	24
6.7	Entrenamiento del árbol en el que se aprecian los nodos pendientes.	27
6.8	Ventana informativa en la que se dibuja la gráfica de la métrica basada en la entropía para cada atributo.	28

6.9	Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.	30
6.10	Vista del árbol con el botón para retroceder al paso anterior.	31
6.11	Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.	33
6.12	Primera arquitectura visual de la red neuronal, formada por capas, neuronas y conexiones densas entre capas consecutivas.	35
6.13	Menú de creación de la red neuronal con selectores para las funciones de activación de las capas.	36
6.14	Selección de columnas objetivo para la red neuronal a partir de un dataset cargado desde un archivo CSV.	39
6.15	Red neuronal durante el entrenamiento, con nombres procedentes del dataset en las neuronas de entrada y salida, y con las columnas visuales de entrada, salida y valor esperado.	40
6.16	Panel de entrenamiento de la red neuronal con controles para avanzar con distintos niveles de granularidad.	42
6.17	Mockup del menú principal realizado con Balsamiq antes de implementar la reorganización final de la entrada a la aplicación.	48
6.18	Versión final del menú principal, mostrando seleccionado el árbol de decisión.	48
6.19	Versión final del menú principal, mostrando seleccionada la red neuronal	49
6.20	Menu de navegacion que permite volver al menú principal y guardar y cargar estructuras de los algoritmos.	50
6.21	Mockups de las disposiciones horizontal y vertical de las vistas de algoritmo realizados con Balsamiq.	50
6.22	Creación de la red tras el menú traslúcido.	51
6.23	Creación de la red al lado del menú de creación.	52
6.24	Imagen de la red en la que los colores de las conexiones son gradientes de avanzan desde el cyan hasta el verde para los valores positivos y del amarillo al rojo para los negativos.	53
6.25	Árbol de decisión en medio del proceso de evaluación del dataset de frutas.	57
7.1	Primeros nodos de la estructura final de un árbol de decisión obtenido con un dataset sintético desde un script externo.	61
7.2	Árbol de decisión generado por la aplicación con el mismo dataset sintético usado en la referencia externa.	63
7.3	Red neuronal entrenada dentro de la aplicación, con los pesos y el sesgo de la neurona de salida visibles en la interfaz.	63

7.4	Lectura externa del archivo ONNX exportado desde la aplicación, con los pesos y el sesgo de la red entrenada.	63
7.5	Red sintética exportada a ONNX desde un entorno externo, con pesos y sesgo conocidos.	64
7.6	Red sintética importada en la aplicación desde ONNX, con los pesos y el sesgo esperados visibles en la interfaz.	64

Índice de cuadros

Capítulo 1

Introducción

PRIMER capítulo da memoria, onde xeralmente se exporán as liñas mestras do traballo, os obxectivos, etc. Incluimos un par de exemplos de citas e de referencias

LA inteligencia artificial ...?

1.1 Contexto y motivación

Los algoritmos de aprendizaje máquina son técnicas matemáticas que escapan de las fronteras de lo que es entendible a simple vista por ¿una persona cualquiera.¿ Desde sus raíces más puramente matemáticas hasta su implementación tanto hardware como software.

Esto ya era cierto en los tiempos de los primeros algoritmos de aprendizaje ¿por refuerzo, árboles o qué?, pero con el surgir de los modelos de aprendizaje profundo, esta realidad es más palpable que nunca.

Bajo estas premisas es que se siente indispensable ¿proporcionar a la gente los materiales y métodos para que puedan comprender lo mejor posible esta tecnología cada vez más omnipresente¿. ¿Y a de más el aprendizaje es mejor si es interactivo ¿?

1.1.1 Definición del Problema

Hoy en día las principales formas de aprender cómo funcionan los modelos de inteligencia artificial (sobre todo los clásicos, pero también los modernos) es a través de los siempre confiables libros y universidad, las más ancestrales fuentes de conocimiento. Entre los formatos más actuales también encontramos los vídeos divulgativos en plataformas online como Youtube, donde la posibilidad de que todo el mundo comparta su contenido ha dado lugar a ¿muchas oportunidades de ver las interpretaciones visuales de otras personas¿.

Sin embargo ¿aún no existe ninguna herramienta pedagógica que permita interactuar en tiempo real con los algoritmos.¿ ¿Programándolos se pueden ver sus resultados, su evolución en una gráfica, pero no palpar los que está sucediendo en las entrañas de la bestia mientras

mastica el equivalente a su comida: los datos, tanto en entrenamiento como en inferencia;?
((.....))

1.1.2 Objetivos

((Crear la aplicación - paso 1, hacerla - paso 2, exportarla
Publicarla para que todo el mundo la pueda disfrutar))
((creo qe hay q se rmás concreto))

Los objetivos que se plantean en este trabajo comienzan por plantearse qué algoritmos de IA son los más interesantes para ¿explicar¿. Esta es, en última instancia, una cuestión subjetiva, por lo que en este trabajo se abordó esta decisión mirando ¿de cara¿ a dos de los mayores paradigmas de la IA: la simbólica y el deeplearning. ((no se si decir de forma pragmática y decir que tenía en mente las redes neuronales, y que los árboles se parecían lo suficiente, o poner un poco de fantasía de "los árboles, uno los primeros amigos del hombre en cuánto a lo que hacer que una máquina aprenda a jugar al ajedrez..." (lo

De ambos se decidió que los mejores candidatos para protagonizar una explicación que busque asentar los fundamentos eran las versiones más fundamentales de cada uno. Del lado de la IA simbólica está el árbol de decisión ((plantado por bla bla bla en blabal)), conocido por ser ... white box Y en el deeplearning, el ladrillo fundacional, la red neuronal ((lo mismo de histoia...))

1.1.3 Alcance y Limitaciones

1.1.4 Metodología de Trabajo

1.1.5 Estructura de la Memoria

Estado del Arte

LA inteligencia artificial y el aprendizaje automático se apoyan en algoritmos cuyo funcionamiento interno no siempre resulta fácil de visualizar. Conceptos como el cálculo de una frontera de decisión o la actualización iterativa de los parámetros de un modelo pueden resultar abstractos cuando se estudian únicamente desde una formulación teórica. Esta dificultad se acentúa en el aprendizaje profundo, donde intervienen elementos como funciones de activación, pesos, sesgos, propagación hacia delante, retropropagación o gradientes. Por ello, se han desarrollado recursos educativos orientados a hacer más comprensible el comportamiento de estos modelos mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos, la relación entre sus componentes internos y la forma en la que una decisión algorítmica se transforma en un resultado observable.

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de vídeos *Neural networks* [1]. Esta fue publicada en el canal de YouTube *3Blue1Brown*. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada, como se aprecia en la Figura 2.1.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red. También es adecuado para motivar y contextualizar el problema gracias a una explicación apoyada en recursos visuales claros. Su formato, en cambio, conserva un carácter fundamentalmente expositivo: en la página web, dos de las diez lecciones incorporan un componente interactivo, y dicha interacción se orienta siempre a la relación entre la entrada y la salida de una red ya entrenada. El estudiante trabaja con una explicación limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del modelo. Esta

restricción deja abierta la necesidad de recursos más activos, en los que sea posible analizar el efecto de cada decisión de diseño sobre el comportamiento de una red neuronal.

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es *A Neural Network Playground* [4], una aplicación que permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento. En la Figura 2.2 se puede visualizar la página web.

A diferencia de los recursos puramente expositivos, la relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Modificar directamente la arquitectura del modelo (número de capas y neuronas) o sus hiperparámetros (tasa de aprendizaje y regularización) altera de manera inmediata el proceso de ajuste. Este enfoque interactivo facilita que el estudiante relacione conceptos críticos como el sobreajuste, la generalización o la convergencia con resultados palpables. La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Esta misma orientación hacia la experimentación aparece en herramientas centradas en arquitecturas más específicas. *GAN Lab* [5] permite experimentar en el navegador con redes generativas adversarias. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados. La aportación principal de GAN Lab es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que existen modelos de aprendizaje profundo con objetivos y esquemas de aprendizaje diferentes.

La limitación principal de *GAN Lab*, junto con la falta de detalle sobre ciertas operaciones internas, es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, *GAN Lab* es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada en los fundamentos de una red neuronal básica.

En el ámbito de los algoritmos clásicos de aprendizaje automático, también existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario

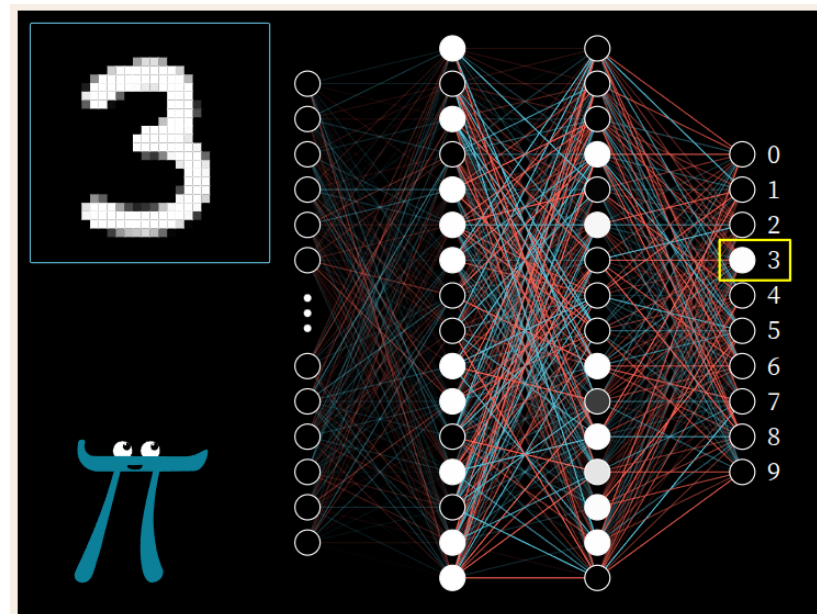


Figura 2.1: Sección interactiva de la página web de 3Blue1Brown.

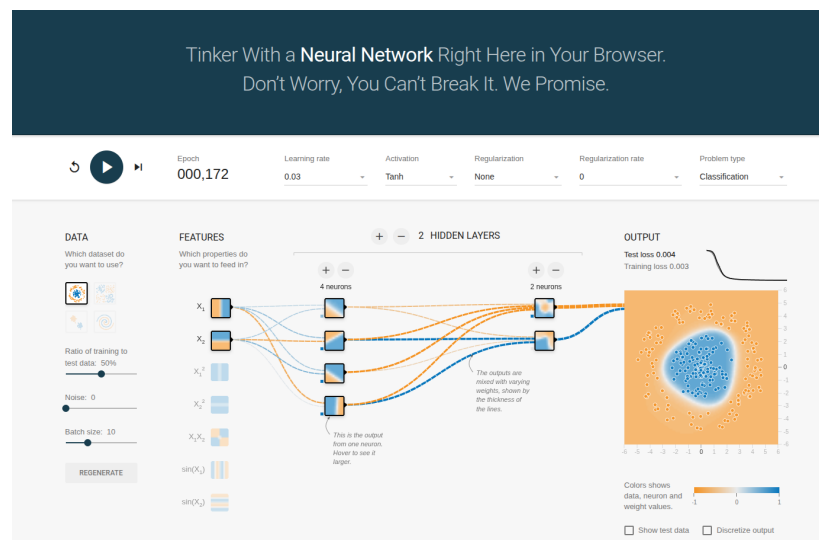


Figura 2.2: Página web TensorFlow Playground.

construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real. Esta propuesta resulta interesante porque acerca al estudiante a la estructura jerárquica del modelo y al proceso de partición sucesiva de los datos. Su experiencia de uso, no obstante, es limitada y las explicaciones que acompañan a la simulación tienen poco desarrollo.

Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión es más reducida. La revisión realizada muestra una presencia mucho mayor de simuladores y visualizaciones dedicados a redes neuronales, tanto en recursos divulgativos como en herramientas web orientadas a la experimentación. Esta diferencia es especialmente visible cuando se buscan recursos que combinen visualización del modelo, explicación del criterio de partición y evaluación paso a paso.

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo. También debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo. En el caso de los árboles de decisión, la visualización debería mostrar cómo se selecciona cada atributo, cómo se divide el conjunto de datos y cómo se alcanza una clasificación final a partir de las características de entrada. En ambos modelos, esta visualización debe ir acompañada de explicaciones textuales que interpreten las métricas, los cálculos y las decisiones del algoritmo, de forma que la simulación no dependa únicamente de la observación visual.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. La elección tecnológica del entorno de desarrollo tuvo en cuenta tanto criterios de implementación como la naturaleza del problema abordado: explicar modelos de aprendizaje automático mediante visualizaciones dinámicas, animaciones y ejemplos manipulables por el usuario. En consecuencia, se priorizaron tecnologías que facilitasen la construcción de interfaces interactivas, la validación de los algoritmos implementados y la publicación del resultado final en un entorno accesible. Godot permite exportar un mismo proyecto a distintas plataformas, como escritorio, móvil o web. En este caso, la versión web se escogió por su accesibilidad: el usuario puede abrir la aplicación desde un navegador, sin instalar software ni preparar un entorno de ejecución específico.

3.1 Godot como motor de desarrollo

La herramienta principal empleada fue Godot, un motor de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas [7]. Sus características resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo. El proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de Escenas, Nodos y Señales que propone Godot [8].

Los Nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un Nodo. Y cada Nodo cuenta, principalmente, con una serie de atributos, métodos y Señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las Señales son disparadores de funciones. Cuando son emitidas llaman

a la función a la que están conectadas. Son una forma de conectar eventos entre diferentes Nodos. Son el núcleo del paradigma de programación que promueve el motor: la programación dirigida por eventos.

Los atributos, métodos y Señales de cada Nodo vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el *Node* una clase vacía, la raíz del resto de Nodos, que solo cuenta con los métodos mínimos para que funcione en el motor; el *Node2D* característico por contar con atributos referentes a la posición en un espacio bidimensional; el *Node3D* similar al anterior pero en un espacio tridimensional; y el *Control* pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

Los Nodos por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las Escenas. Estas son la materialización concreta del proyecto: los archivos que son guardados en disco y con los que se interactúa en el editor como se puede ver en la Figura 3.1. Están formadas, como mínimo, por un Nodo raíz. De este Nodo raíz pueden colgar Nodos hijo, que a su vez pueden contar con sus propios Nodos hijo, y así sucesivamente.

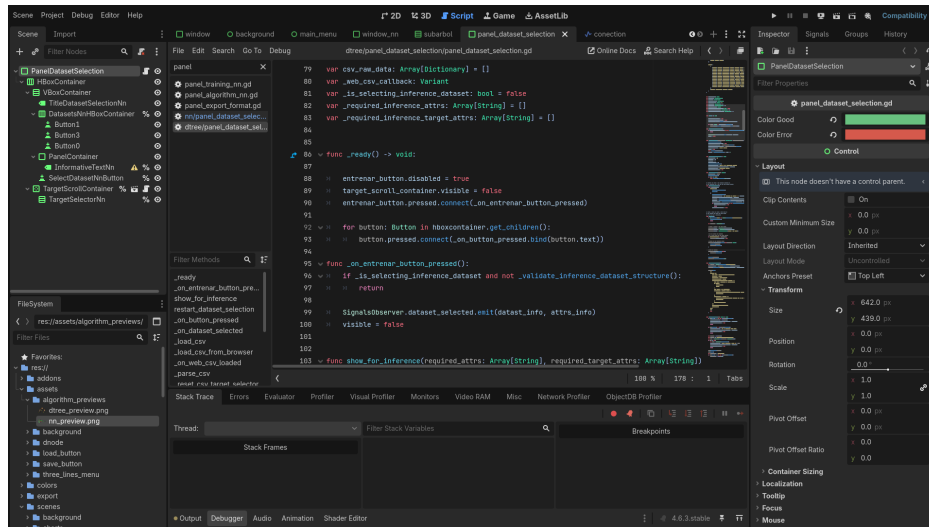


Figura 3.1: Entorno de desarrollo de Godot.

Finalmente, las Escenas, como en última instancia son árboles de Nodos, pueden ser instanciadas dentro de otras Escenas. Esto habilita una modularidad total, puesto que toda estructura de Nodos que sea susceptible de ser reutilizada, puede guardarse como una Escena para no tener que ser recreada.

Además, los Nodos pueden tener asociado un script. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del Nodo al que están asociados, así

como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al Nodo raíz y, desde este, modificar cualquier propiedad global o acceder a cualquier Nodo en la Escena. Además, mediante la creación de *Autoloads* en el proyecto también se puede acceder a clases personalizadas en cualquier punto del código.

La Figura 3.2 muestra de forma esquemática esta organización y sitúa las Escenas, los Nodos, los Scripts y los módulos dentro de la arquitectura general del motor.

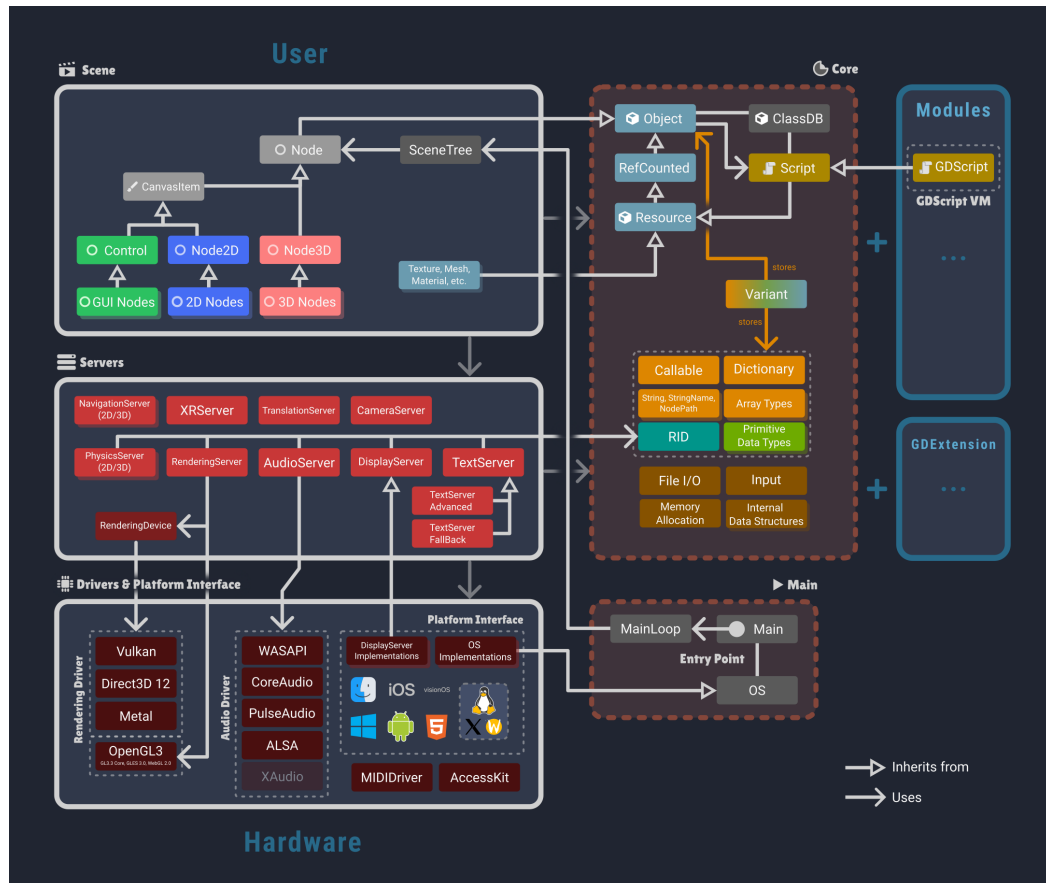


Figura 3.2: Diagrama general de la arquitectura de Godot.

La organización en Escenas permite separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor acoplamiento. El sistema de Señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

La manera en que Godot organiza las interfaces mediante Nodos permite estructurar los

elementos de los algoritmos tratados de forma sencilla. Un árbol de decisión puede mostrarse como una estructura jerárquica de Nodos conectados, mientras que una red neuronal puede representarse mediante capas, neuronas y conexiones. Esta correspondencia conceptual ayuda a trasladar las estructuras internas del algoritmo a elementos visuales manipulables dentro de la aplicación.

Otro motivo importante para escoger Godot fue su sistema de exportación multiplataforma. El motor permite generar versiones para diferentes entornos a partir del mismo proyecto, incluida una versión HTML5 de la aplicación [9]. La exportación web se adoptó por la forma en la que se quería distribuir la herramienta: una aplicación educativa debe ser sencilla de abrir, compartir y probar. Publicarla como aplicación web evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

3.2 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [10]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental. Esto resultó útil durante el desarrollo, ya que la aplicación fue creciendo mediante pruebas sucesivas: primero se validó la interacción básica, después se incorporaron los algoritmos y más adelante se añadieron explicaciones y mejoras de interfaz. Disponer de un historial de cambios permitió avanzar con mayor seguridad y conservar puntos estables del proyecto mientras se realizaban modificaciones.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [11]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La aplicación exportada se publica como un conjunto de archivos preparados para ejecutarse en el navegador, distinto del proyecto editable de Godot.

Esta fase tiene importancia porque marca la diferencia entre una aplicación que funciona dentro del entorno de desarrollo y una herramienta que puede utilizar cualquier persona desde un enlace. En el editor, el proyecto se ejecuta en un contexto controlado, con acceso directo a las Escenas, Scripts y recursos locales. En el navegador, la aplicación depende de los archivos exportados, de las restricciones propias de la plataforma web y de la forma en la que el servicio de publicación sirve esos archivos. Por este motivo, el despliegue se trató como una parte más del desarrollo y no únicamente como el último paso administrativo.

La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consul-

tarse desde el navegador sin infraestructura de servidor adicional. Esto encaja con el objetivo de accesibilidad planteado al escoger la exportación web: el usuario solo necesita abrir una dirección, mientras que el proyecto mantiene un proceso de publicación suficientemente simple para ser actualizado durante el desarrollo.

Metodología y planificación

Marco teórico

Este capítulo recoge la base formal de los dos modelos implementados en la aplicación. En primer lugar se presentan los árboles de decisión contruidos mediante ID3 con atributos categóricos y entropía. Después se describe el funcionamiento de una red neuronal multicapa sencilla, formada por capas densas y entrenada mediante retropropagación.

5.1 Árboles de decisión e ID3

Los árboles de decisión son modelos supervisados que organizan una clasificación como una secuencia de preguntas sobre los atributos de entrada. Cada nodo interno representa una pregunta, cada rama representa una posible respuesta y cada hoja contiene la etiqueta predicha. El algoritmo ID3 fue presentado por Quinlan como un procedimiento para inducir árboles de decisión a partir de ejemplos etiquetados [12]. La versión considerada en este trabajo usa atributos categóricos y selecciona las divisiones mediante entropía y ganancia de información.

Sea D un conjunto de ejemplos de entrenamiento y sea C el conjunto de clases posibles. Para una clase $c \in C$, se define

$$p(c) = \frac{|\{x \in D : y(x) = c\}|}{|D|},$$

donde $y(x)$ es la etiqueta del ejemplo x . La entropía del conjunto D mide la incertidumbre sobre la clase de un ejemplo tomado de ese conjunto:

$$H(D) = - \sum_{c \in C} p(c) \log_2 p(c).$$

Cuando $p(c) = 0$, el término $p(c) \log_2 p(c)$ se toma como 0. Si todos los ejemplos pertenecen a la misma clase, la entropía vale 0. Si las clases aparecen repartidas de forma más equilibrada, la entropía aumenta.

Para evaluar un atributo categórico A , ID3 separa D en subconjuntos según los valores posibles de ese atributo. Si $\mathcal{V}(A)$ es el conjunto de valores de A , se define

$$D_v = \{x \in D : A(x) = v\}, \quad v \in \mathcal{V}(A).$$

La entropía esperada tras dividir por A es la media ponderada de las entropías de esos subconjuntos:

$$H(D | A) = \sum_{v \in \mathcal{V}(A)} \frac{|D_v|}{|D|} H(D_v).$$

La ganancia de información indica cuánto se reduce la incertidumbre al usar ese atributo:

$$IG(D, A) = H(D) - H(D | A).$$

ID3 escoge el atributo con mayor ganancia de información:

$$A^* = \arg \max_{A \in \mathcal{A}} IG(D, A),$$

donde $\mathcal{A}$ es el conjunto de atributos disponibles en ese nodo.

El algoritmo se aplica de forma recursiva. En cada nodo se comprueba primero si todos los ejemplos tienen la misma clase. En ese caso se crea una hoja con esa etiqueta. Si todavía hay varias clases y quedan atributos disponibles, se escoge el atributo con mayor ganancia de información y se crea una rama para cada valor observado. Después, el mismo proceso se repite sobre cada subconjunto D_v , retirando el atributo usado para que no vuelva a seleccionarse en las ramas inferiores. Si se agotan los atributos antes de conseguir subconjuntos puros, se crea una hoja con la clase mayoritaria del conjunto que llega a ese nodo:

$$\hat{c} = \arg \max_{c \in C} |\{x \in D : y(x) = c\}|.$$

La formulación anterior corresponde a la versión básica de ID3 usada en la aplicación. Existen extensiones y variantes que modifican varios puntos del proceso: se pueden tratar atributos continuos mediante umbrales, utilizar otras métricas como el índice de Gini o la razón de ganancia, aplicar técnicas de poda para reducir sobreajuste o permitir árboles con restricciones distintas sobre su profundidad. En este trabajo se fija el caso categórico con entropía porque permite mostrar de forma directa cómo una métrica numérica justifica cada partición del conjunto de datos.

5.2 Redes neuronales multicapa

Las redes neuronales artificiales tienen su origen en modelos como el perceptrón de Rosenblatt [13]. La versión utilizada en este trabajo corresponde a un perceptrón multicapa, es decir, una red formada por una capa de entrada, cero o más capas ocultas y una capa de salida. El entrenamiento de redes multicapa mediante retropropagación fue popularizado por Rumelhart, Hinton y Williams [14], y constituye la base del procedimiento implementado en la aplicación.

Una red neuronal densa se organiza en capas numeradas desde 0 hasta L . La capa 0 es la capa de entrada y contiene el vector de atributos:

$$a^{(0)} = x.$$

Para cada capa posterior $l \in \{1, \dots, L\}$, el modelo dispone de una matriz de pesos $W^{(l)}$, un vector de sesgos $b^{(l)}$ y una función de activación f_l . El cálculo de la capa se divide en dos pasos. Primero se obtiene la combinación lineal:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}.$$

Después se aplica la función de activación:

$$a^{(l)} = f_l(z^{(l)}).$$

El vector $a^{(l)}$ pasa a ser la entrada de la capa siguiente. La salida final de la red es $a^{(L)}$.

Las funciones de activación introducen transformaciones no lineales o adaptan la salida al tipo de problema. Entre las funciones habituales se encuentran la identidad, usada a menudo en regresión,

$$f(z) = z,$$

la sigmoide,

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

y ReLU,

$$\text{ReLU}(z) = \max(0, z).$$

En clasificación multiclase se usa con frecuencia softmax, que transforma un vector de valores en una distribución de probabilidades:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}}.$$

Softmax se aplica al vector completo de la capa, ya que cada salida depende de todos los componentes del vector z .

Para entrenar la red hace falta definir una función de pérdida L , que mide la diferencia entre la salida calculada $a^{(L)}$ y el objetivo esperado y . En regresión puede emplearse el error cuadrático medio, expresado para una muestra como

$$L = \frac{1}{2} \|a^{(L)} - y\|^2.$$

En clasificación con softmax suele usarse entropía cruzada:

$$L = - \sum_i y_i \log a_i^{(L)}.$$

El entrenamiento busca ajustar los pesos y sesgos para reducir esta pérdida.

La retropropagación calcula cómo afecta cada parámetro a la pérdida mediante la regla de la cadena. Para ello se define un término de error o delta en cada capa:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}.$$

En la capa de salida, cuando se combina softmax con entropía cruzada, aparece una simplificación muy usada:

$$\delta^{(L)} = a^{(L)} - y.$$

Para otras combinaciones de pérdida y activación, el delta de salida se obtiene multiplicando el gradiente de la pérdida respecto a la activación por la derivada local de la activación:

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot f'_L(z^{(L)}),$$

donde $\odot$ indica producto elemento a elemento.

En una capa oculta, el error se obtiene a partir de los deltas de la capa siguiente:

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot f'_l(z^{(l)}).$$

Esta expresión resume la contribución de la capa l al error posterior y la ajusta según la pendiente local de su función de activación.

Una vez conocidos los deltas, los gradientes de los parámetros de cada capa quedan definidos como

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T,$$

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

Con descenso de gradiente estocástico, los parámetros se actualizan tras cada muestra con una tasa de aprendizaje η :

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}},$$
$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}.$$

Este ciclo se repite sobre las muestras de entrenamiento. Cada muestra produce una propagación hacia delante, una pérdida, una retropropagación de deltas y una actualización de parámetros.

Durante la inferencia se utiliza únicamente la propagación hacia delante. Los pesos y sesgos permanecen fijos, la entrada atraviesa las capas mediante las mismas ecuaciones y la salida final se interpreta como predicción del modelo.

Capítulo 6

Desarrollo

este capítulo ha quedado bastante informal desde dónde hasta dónde es "este capítulo"?

En este capítulo se describe el proceso seguido para construir la aplicación. La explicación se plantea como una evolución por versiones. Comienza por comprobar que la idea es viable. Después se desarrolla el árbol de decisión, desde sus inicios hasta el resultado funcional con todas las características que se buscaban. Continúa con el desarrollo de la red neuronal, pasando por todas sus fases de creación y mejora. Y termina con el pulido final de la interfaz y retoques a todas las funcionalidades con las que cuenta el resultado final. [Decir que el desarrollo se hace siguiendo scrum y plantear aquí una tabla con prototipos y características?](#) (igual aprovechar y borrar la sección de metodología?) Lo comentamos en la reunión

6.1 Prototipo: versión 0.1

El primer objetivo consiste en comprobar que la herramienta cualifica para crear una aplicación interactiva y publicarla en una página web. Antes de invertir más tiempo en el desarrollo, es necesario verificar tres aspectos básicos: que Godot permite crear una interfaz sencilla, que se puede representar una estructura con forma de árbol y que el resultado puede ejecutarse desde el navegador.

Por ese motivo, la primera versión del proyecto es deliberadamente simple. [¿Se ha decidido implementar un mockup para verificar que la herramienta seleccionada cumple con todos los requisitos? ya..](#) Consiste en un menú inicial y en una Escena donde el usuario puede poner y quitar nodos que se colocan formando una estructura de árbol. Al pulsar sobre los nodos aparece una opción para añadir un nodo hijo debajo, que se organiza jerárquicamente junto al resto. Esta prueba está todavía lejos de un árbol de decisión real. Su valor está en comprobar si la interacción principal de la aplicación no da problemas.

6.1.1 Primeras Escenas

La primera Escena creada es el menú principal. *no se dónde aclarar que Escenas y Nodos son los de godot y escena y nodo significan lo que ponga en la rae Yo al inicio del desarrollo pondría una pequeña frase en plan: en este trabajo escena significa esto y nodo significa esto del desarrollo? Entiendo el punto, y pensé en poner algo, pero se me hace desordenado.* En ese momento tiene un único botón, ya que su papel es servir como punto de entrada hacia la prueba del árbol. Incluir este menú desde el principio ayuda a pensar la aplicación como un conjunto de módulos y facilita su ampliación posterior. En la Figura 6.1 se ve esta primera versión.

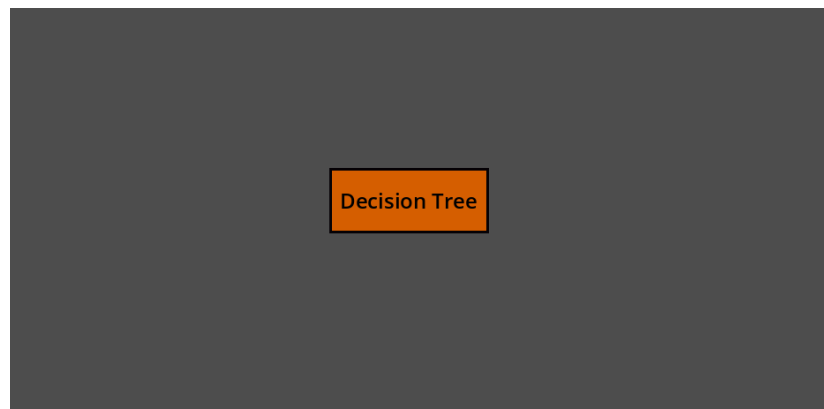


Figura 6.1: Versión preliminar del menú principal.

Al entrar en la escena del árbol aparece un único elemento inicial. Desde él se pueden crear nuevos elementos hijos, conectados por líneas y colocados bajo el anterior. También se añade la posibilidad de borrar partes del árbol, con el fin de comprobar que la estructura puede cambiar durante la ejecución sin que la escena deje de ser comprensible.

Esta primera prueba tiene una finalidad muy concreta: validar la interacción. Lo importante es comprobar que el usuario puede construir una jerarquía visual, que las conexiones se actualizan y que la pantalla sigue siendo manejable al crecer el número de elementos. En la Figura 6.2 se muestra cómo era esta Escena inicial. *Esta imagen igual hay que forzarla a aparecer aquí, la verdad es que ahora mismo la renderiza lejisimos latex veo si se puede. Tendría más sentido que todas las imágenes estuvieran dnd se mencionan en vd. Quitando la flag "p" pasan cosas.* También se dejan visibles dos botones relacionados con cargar y guardar.

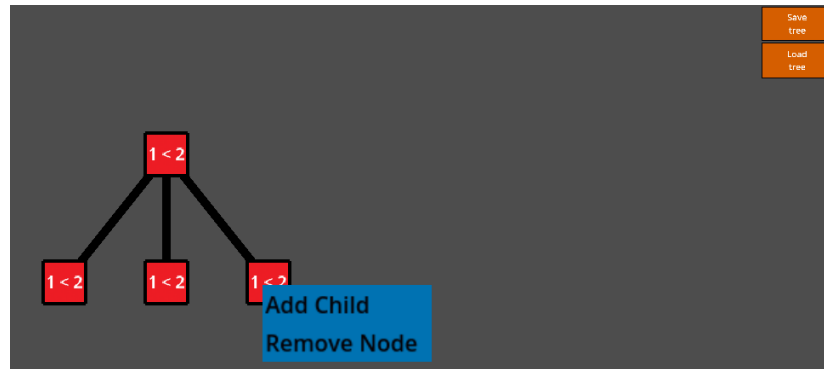


Figura 6.2: Primera Escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda se ve una sencilla estructura con forma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.

6.1.2 Exportación web con GitHub Pages

Una vez verificado el funcionamiento de la prueba interactiva en el entorno de desarrollo de Godot, se procedió a evaluar su despliegue en un navegador web. Esta fase resulta fundamental, ya que la aplicación está concebida para ser utilizada a través de una interfaz web y, por tanto, es necesario garantizar su correcta ejecución fuera del editor.

La exportación del proyecto al formato web se realizó utilizando las herramientas proporcionadas por Godot. Posteriormente, la aplicación generada se publicó mediante GitHub Pages, configurando el repositorio para servir los archivos exportados como una página web accesible públicamente.

La realización de esta prueba permitió validar la viabilidad del flujo de trabajo previsto para el proyecto, que comprende el desarrollo en Godot, la exportación al entorno web y su posterior publicación. Asimismo, la verificación temprana de este proceso contribuyó a reducir riesgos técnicos asociados al despliegue, permitiendo confirmar la adecuación de la tecnología seleccionada antes de abordar etapas posteriores del desarrollo.

Añado aquí también algo de chicha sobre la exportación al ghpages?

6.2 Árbol de decisión: versión 0.2

A partir de esta iteración se incorporó el primer algoritmo de aprendizaje a la aplicación: un árbol de decisión basado en ID3. El objetivo consistió en representar gráficamente la construcción del árbol, visualizar su evolución paso a paso y mostrar las decisiones tomadas durante el proceso de entrenamiento. La dificultad principal está en coordinar tres niveles: el algoritmo que calcula la siguiente división, la estructura lógica del árbol y la escena que el

usuario ve en pantalla. El funcionamiento formal de ID3 y del criterio de entropía se recoge en la Sección 5.1; en este capítulo se describe cómo se llevó ese algoritmo a la aplicación.

6.2.1 Primera arquitectura del árbol

El primer cambio consiste en preparar una Escena que pueda albergar un árbol construido por el algoritmo sin importar su tamaño. *No se si se entiende esta frase okey, la cambio. Mejor?* La prueba inicial servía para validar que se podían añadir y borrar nodos manualmente, aunque su estructura resultaba insuficiente para un entrenamiento real. Un árbol de decisión puede crecer varios niveles y abrir muchas ramas en poco espacio, por lo que una pantalla fija deja de ser útil en cuanto el número de nodos aumenta. Por esta razón se incorpora una zona desplazable dentro de la interfaz, de manera que el usuario pueda moverse por el árbol cuando su tamaño supere el área visible. *aquí no se si mencionar o no lo de que uso un Nodo Control de Godot de tipo ScrollContainer. Supongo que no, pero por si acaso.*

También se revisa la distribución de los nodos en el árbol. En la versión manual bastaba con colocar cada hijo bajo su padre, ya que el objetivo era comprobar la interacción. Durante el entrenamiento algorítmico se necesita una disposición más estable: los hijos deben aparecer por debajo, el padre debe quedar centrado respecto a ellos y el árbol debe reorganizarse cada vez que se añade una rama. Para conseguirlo se introduce un cálculo de posiciones basado en la profundidad de cada nodo y en su orden dentro de la estructura. Así se evita que las ramas se acumulen en la misma zona y se mantiene una lectura jerárquica clara como se aprecia en la Figura ?? *Yo aquí enseñaría el resultado pero, cómo? Mostrando que antes se hacía mal y ahora bien?*

Otra decisión importante es separar la información lógica del árbol de su representación visual. Cada nodo visible conserva un identificador, una referencia al padre, una lista de hijos, su profundidad y la información que representa, como el atributo de decisión o la etiqueta final. *a lo mejor sobra esta lista? la pongo y si sobra se quita* Al mismo tiempo, el árbol mantiene un diccionario interno que permite localizar cada nodo por su identificador. Esta duplicidad resulta útil porque la Escena de Godot sirve para dibujar, animar y recibir eventos, mientras que el diccionario permite recorrer la estructura, recalcular posiciones, eliminar subárboles y evaluar ejemplos más adelante.

Las conexiones entre nodos se tratan como elementos propios. Cada conexión representa una rama del árbol y puede mostrar el valor del atributo que conduce desde el nodo padre hasta el hijo. Esta separación permite que los nodos se encarguen de mostrar decisiones o clasificaciones, y que las conexiones expliquen el camino seguido. Más adelante, esta decisión facilita tanto la animación de las ramas como la evaluación visual de nuevos datos.

Por ejemplo, en esta frase de aquí yo creo que el presente continuo queda mal, y que habría que poner más bien "se añadió el modo automático" estas seguro de que no se exige que el tex-

to este escrito en impersonal? ... no lo he leído en ninguna parte Con la base visual preparada, se añade el modo automático. Este es un estado interno de la Escena del árbol diferenciado del modo manual implementado hasta ahora. Como se vió hasta ahora, en el modo manual el usuario interactuaba directamente con los nodos, pudiendo decidir si añadirlos o quitarlos y dónde hacerlo. Pero en el nuevo modo automático es el algoritmo ID3 en base a un dataset el que se encarga de marcar el crecimiento del árbol. lo de que llama a diferente código no se puede poner así. Di que hay dos versiones manual y automatica y como funcionan textcolor-roliveready, a ver si mejor En este modo, el usuario controla el avance del entrenamiento en lugar de añadir ramas manualmente. Yo no diría que es sencilla okey La primera versión de la interfaz cuenta con un botón para iniciar el proceso y con otro para avanzar al siguiente paso, como se muestra en la Figura 6.3.

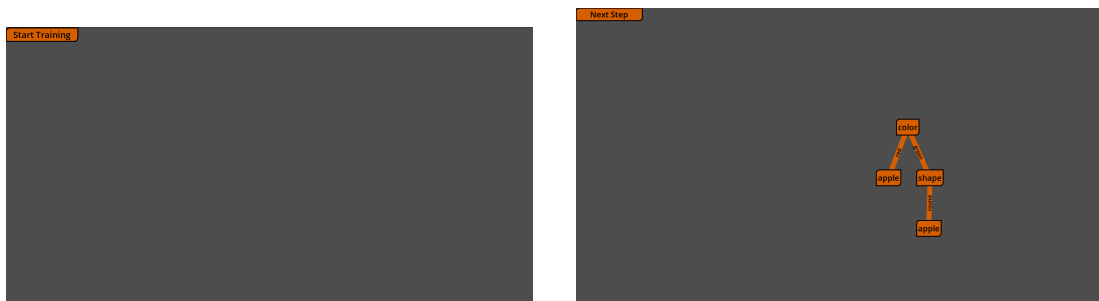


Figura 6.3: Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.

La construcción paso a paso se apoya en una separación de responsabilidades. El algoritmo calcula qué tipo de nodo debe crearse en cada momento y emite la información necesaria para hacerlo: padre, valor de la rama, atributo elegido, etiqueta si se trata de una hoja y datos explicativos del paso. El controlador recibe esa información y la traduce a cambios en la Escena, creando o actualizando los nodos visibles. De este modo, la lógica de aprendizaje queda aislada de los elementos gráficos de Godot, y la vista puede centrarse en representar el resultado de cada decisión.

Para que el usuario pueda avanzar con el botón de siguiente paso e ir comprendiendo el proceso, el algoritmo no construye todo el árbol de una sola vez. El trabajo pendiente se guarda como una serie de contextos, cada uno con los datos que llegan a una rama, los atributos disponibles, el padre visual y la profundidad correspondiente. Cada pulsación extrae uno de esos contextos, decide si debe convertirse en una hoja o en un nodo interno y actualiza la Escena. Esta organización incremental ayuda a que el árbol pueda explicarse nodo a nodo sin necesidad de tener que almacenar toda la información a la que se quiere acceder en memoria.

Las primeras pruebas se realizan con un conjunto de datos pequeño y controlado. Su fina-

lidad es comprobar que el algoritmo crea la raíz, abre ramas, coloca los nodos en posiciones con sentido y mantiene sincronizados el estado interno y la vista. En esta fase queda fijado el patrón principal del resto del desarrollo del árbol: el algoritmo decide, el controlador traduce esa decisión y el árbol visible se reorganiza tras cada paso.

6.2.2 Reorganización de la Escena y primeros elementos explicativos

Cuando el modo automático empieza a funcionar, la Escena se reorganiza para reducir elementos provisionales. La vista del árbol pasa a integrarse mejor con los contenedores de interfaz de Godot, lo que permite combinar botones, paneles y zonas desplazables de forma más natural. Esta reorganización también mejora el centrado del árbol. En lugar de mover la escena completa, se recalculan las posiciones internas de los nodos y el área mínima del contenedor desplazable. Así el árbol aparece centrado cuando cabe en pantalla y sigue pudiendo recorrerse cuando crece.

La Escena se convierte en ese momento en un flujo guiado. Al entrar en la vista del árbol se muestra primero un panel que permite escoger entre creación manual y creación algorítmica. Cada opción activa controles distintos y evita mezclar acciones pensadas para usos diferentes. En la Figura 6.4 se muestra este panel inicial.

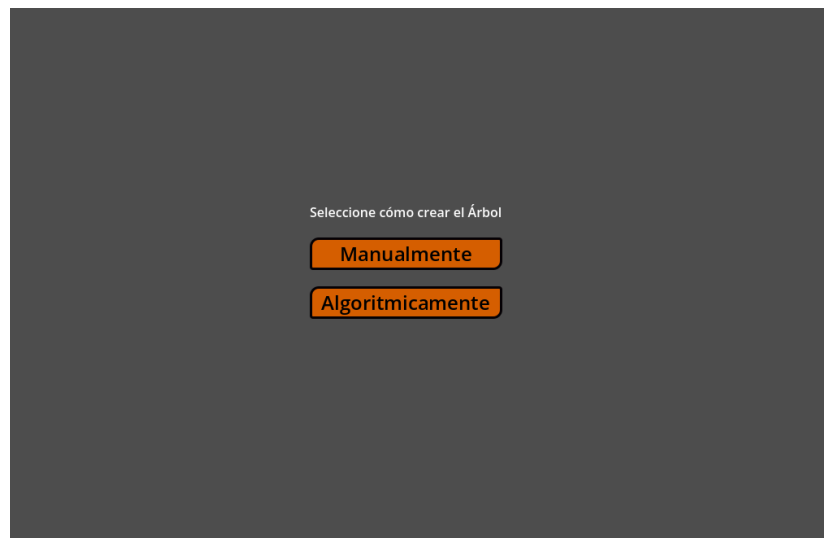


Figura 6.4: Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.

En esta etapa también se mejora la apariencia de los nodos. Los nodos internos, que representan preguntas sobre un atributo, y las hojas, que representan una clasificación final, reciben estilos visuales distintos, como se aprecia en la Figura 6.5. Esta distinción permite interpretar la estructura con mayor rapidez: algunos nodos indican que todavía hay una de-

cisión que tomar, y otros señalan que se ha llegado a un resultado. Además se impide que los nodos creados por el algoritmo sean editados manualmente, ya que representan el resultado del entrenamiento y deben cambiar solo cuando avanza o retrocede el proceso. La interfaz se traduce al castellano en este mismo tramo. Los controles principales pasan a tener escrito el texto “Empezar entrenamiento” y “Siguiente paso” como se aprecia en la Figura 6.5. El cambio ayuda a que la vista del árbol sea coherente con el idioma del resto de la aplicación y con el tono explicativo del trabajo.

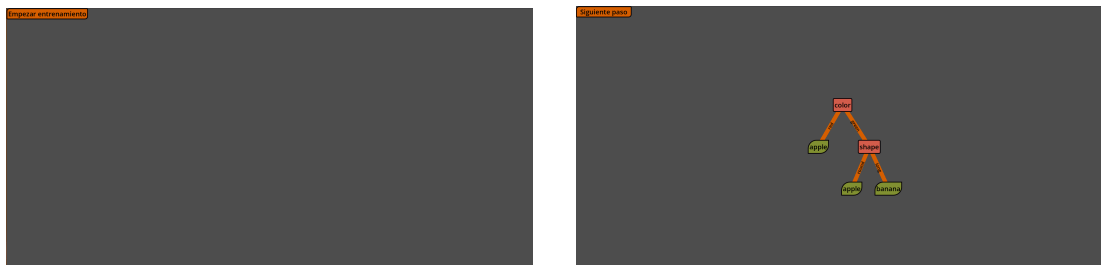


Figura 6.5: Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.

Una vez que la construcción visual resulta estable, se incorpora la ventana informativa. Hasta este punto, la escena permite ver qué se añade al árbol, aunque todavía ofrece poca información sobre el motivo de cada decisión. La ventana nace para acompañar cada paso con una explicación escrita y sincronizada con el estado interno del algoritmo. Su primera versión contiene un mensaje inicial que invita a comenzar el entrenamiento, como se observa en la Figura 6.6.

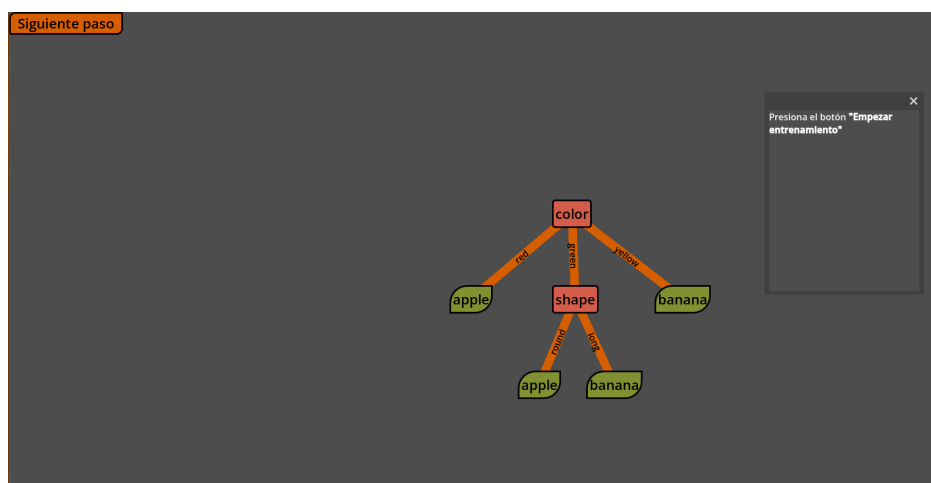


Figura 6.6: Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.

La ventana se implementa como un componente separado de la vista principal. Esta separación permite actualizar su contenido cada vez que el algoritmo procesa un contexto, sin mezclar la redacción de explicaciones con la creación de nodos en pantalla. El algoritmo prepara un conjunto de datos explicativos del paso actual: etiquetas presentes, número de ejemplos, atributos disponibles, tipo de nodo creado, atributo escogido y valores de la métrica usada para comparar alternativas. *Aquí tb listé, y pa adelante* La ventana recibe esa información y la convierte en texto dirigido al usuario.

Los primeros textos cubren tres casos fundamentales. El primero aparece cuando se crea un nodo interno y hay que escoger por qué atributo dividir los datos. El segundo se usa cuando todos los ejemplos que llegan a una rama comparten la misma etiqueta y se puede crear una hoja pura. El tercero explica la creación de una hoja por mayoría, necesaria cuando quedan ejemplos con etiquetas distintas y ya no existen atributos útiles para seguir dividiendo. Pero durante las pruebas se detecta que la información interna del algoritmo necesita una preparación antes de mostrarse. Las listas de etiquetas o atributos, si se presentan tal como las maneja el código, tienen un formato propio de programación y resultan poco naturales en una explicación. Por ello, la ventana transforma esos valores en frases legibles, con enumeraciones y textos adaptados al caso concreto. Esta decisión nace del objetivo de la aplicación, que gira alrededor de que la interfaz debe traducir las estructuras internas del programa a una explicación comprensible para el usuario.

También se empieza a mostrar el criterio utilizado para elegir la siguiente división. Al principio aparece como una lista textual de valores calculados para cada atributo. Aunque es una solución sencilla, ya conecta el cálculo del algoritmo con una justificación visible dentro de la interfaz. A partir de este punto, la construcción del árbol queda acompañada por una justificación de cada paso.

6.2.3 Animación de la construcción del árbol

Cuando la construcción paso a paso está funcionando, se aprecia otro problema de lectura. Una pulsación puede crear un nodo, desplazar parte del árbol y dibujar varias ramas nuevas. El resultado lógico es correcto, aunque el cambio visual puede ser demasiado brusco para seguirlo con claridad. Por esta razón se incorporan animaciones a la construcción.

Las primeras animaciones se aplican a las conexiones. Cada rama se dibuja progresivamente desde el nodo padre hasta el nodo hijo, de manera que el usuario pueda identificar qué parte acaba de aparecer. Las etiquetas de las ramas también se integran en la conexión: la línea deja un espacio para el texto y este se orienta siguiendo la inclinación de la rama, con la corrección necesaria para que no quede invertido. Así se muestra el valor del atributo que conduce a cada hijo sin llenar los nodos de información secundaria.

Después se anima la aparición de los nodos. En vez de mostrarse de golpe, los nodos nue-

vos aparecen mediante una transición de opacidad. Este efecto tiene una finalidad funcional, no solo estética, ya que dirige la atención hacia el elemento incorporado en el paso actual y ayuda a distinguirlo de los nodos que ya estaban presentes.

otro caso donde creo que suefa forzado La introducción de animaciones obliga a revisar la actualización de conexiones. En una versión temprana, al recalcular el árbol se redibujaban todas las ramas. Eso provocaba que una conexión antigua volviese a animarse cada vez que se añadía una nueva. La solución consiste en conservar las conexiones existentes, actualizar su posición si el árbol se recoloca y animar únicamente las conexiones creadas en ese paso. Con ello, el movimiento de la escena comunica mejor qué elemento pertenece al avance actual.

En esta misma etapa se añaden los nodos pendientes. Cuando el algoritmo crea un nodo interno, ya conoce los valores del atributo elegido y, por tanto, las ramas que saldrán de él. Todavía queda por decidir si cada rama terminará en una hoja o en otro nodo de decisión. Hasta este momento, esta incertidumbre no era representada de ninguna forma, ya que las ramas no eran dibujadas hasta que el siguiente nodo era definido. Como el objetivo es representar la información que tiene el árbol en cada instante, se optó por crear nodos temporales con un texto provisional y un estilo propio que los diferencia. **aquí he metido un pretérito que no se si desentona mucho** Así las conexiones son dibujadas cuándo el algoritmo las tiene en cuenta. Los nodos pendientes muestran que una rama ya está prevista y que su contenido se resolverá en un paso posterior. Cuando el algoritmo procesa esa rama, el nodo temporal se transforma en el nodo definitivo, con su atributo o su etiqueta final. La transformación también se anima: el nodo reduce su opacidad, cambia de texto y tema visual, y vuelve a mostrarse. De esta manera, el usuario percibe que se está completando una parte del árbol que ya existía como posibilidad abierta. **No se si es correcto poner estas "afirmaciones" sobre el usuario** En la Figura 6.7 se puede ver el aspecto de estos nodos pendientes.

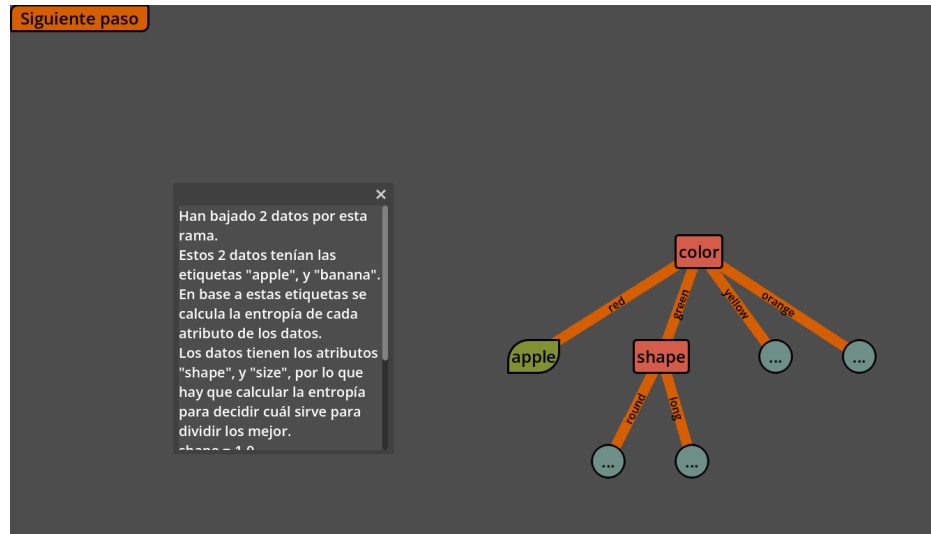


Figura 6.7: Entrenamiento del árbol en el que se aprecian los nodos pendientes.

También se añade una forma de obtener información de los nodos una vez estos han pasado. Al pasar por ciertos nodos, el texto podía cambiar para mostrar un valor numérico asociado a la decisión. La idea permitía consultar información sin recargar visualmente todo el árbol, aunque un número aislado resultaba difícil de interpretar fuera de contexto. Esta prueba ayuda a confirmar que la explicación principal debía concentrarse en la ventana informativa, donde los valores pueden acompañarse de texto y comparaciones visuales.

6.2.4 Gráficas y casos explicativos en la ventana

La ventana informativa evoluciona a medida que los pasos del árbol se vuelven más complejos. Una explicación en texto plano resulta suficiente para los primeros casos, aunque se queda corta cuando el algoritmo compara varios atributos y escoge uno de ellos. El objetivo pasa entonces a combinar texto y apoyo visual, de forma que el usuario pueda ver la comparación en lugar de leer solo una lista de números.

El cambio principal es la incorporación de una gráfica de barras, mostrada en la Figura 6.8. La gráfica representa la métrica calculada para cada atributo, basada en la entropía y utilizada por el algoritmo como ganancia de información, tal como se define en la Sección 5.1. Esta representación permite comparar de un vistazo qué atributo reduce mejor la incertidumbre de los datos y por qué se utiliza para abrir las ramas del nodo actual.

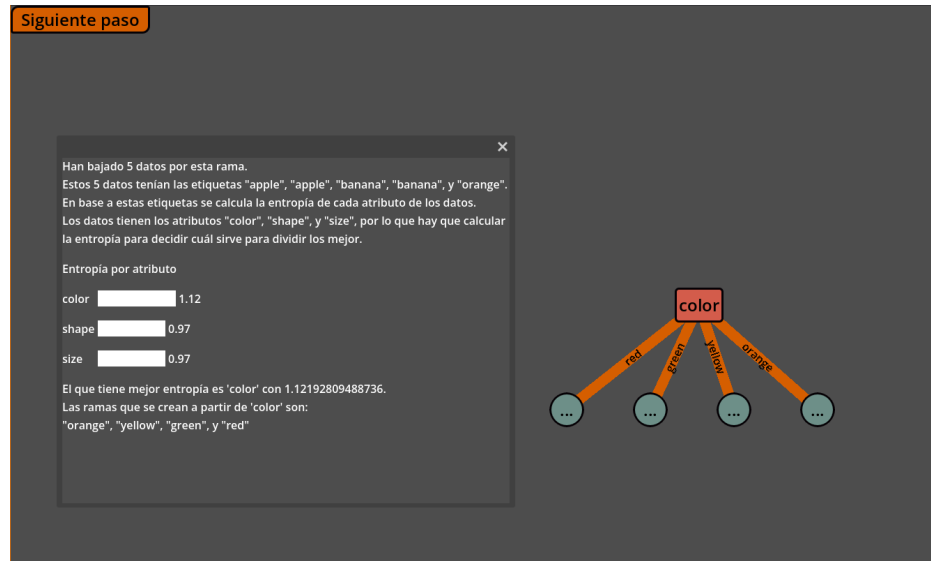


Figura 6.8: Ventana informativa en la que se dibuja la gráfica de la métrica basada en la entropía para cada atributo.

Para integrar la gráfica, la ventana se reorganiza como una zona desplazable con varios elementos. El texto se divide en dos partes: primero se presentan los datos que llegan al nodo, las etiquetas y los atributos disponibles; después se inserta la gráfica; al final se explica el atributo elegido y las ramas que se van a crear. Esta estructura evita que la gráfica aparezca como un elemento aislado y la sitúa dentro del relato del paso actual.

El cambio también modifica la forma en la que se preparan los datos explicativos. Los valores de la métrica dejan de generarse únicamente como frases ya formateadas y se conservan como números. La misma información puede usarse entonces para dos fines: construir el texto de la explicación y alimentar la gráfica de barras. Esta separación mantiene la ventana sincronizada con el cálculo real del algoritmo y evita duplicar la lógica de la métrica en la interfaz.

Durante las pruebas aparece un caso importante: varios atributos pueden obtener el mismo valor máximo de la métrica. Si la aplicación escoge uno de ellos sin explicar la situación, la decisión parece arbitraria. Para cubrir este caso se añade una explicación específica de empate. La ventana indica que hay varias alternativas equivalentes y que el algoritmo continúa con una de ellas para poder seguir construyendo el árbol. También se revisan los textos asociados a las hojas. Las primeras redacciones estaban pensadas para conjuntos muy pequeños, donde a veces solo llegaba un ejemplo a cada resultado final. Al probar datos más variados, se hizo necesario explicar hojas creadas a partir de varios ejemplos con la misma etiqueta y hojas creadas por mayoría. La ventana pasa así a describir el motivo de una clasificación final de forma más general, sin depender del tamaño exacto del subconjunto que llega a la rama.

Este proceso de ajuste se repite durante toda la fase del árbol. Cada nuevo conjunto de datos revela situaciones que la explicación inicial no cubría bien: ramas puras, atributos agotados, empates, varios ejemplos con la misma clasificación o decisiones que crean nodos pendientes. La ventana va incorporando esos casos para que el usuario no tenga que interpretar silencios de la interfaz.

6.2.5 Primer panel de selección de datos

Hasta este momento, los datos de entrenamiento están definidos dentro del propio proyecto. Esta decisión facilita las primeras pruebas, porque permite repetir siempre el mismo ejemplo y centrar el esfuerzo en el algoritmo y la visualización. Cuando la construcción del árbol se estabiliza, esa limitación empieza a ser relevante, ya que es muy interesante que una herramienta didáctica permita observar cómo cambian las decisiones al cambiar los datos.

El primer paso para resolverlo es probar un segundo conjunto preparado manualmente. Este nuevo ejemplo incluye atributos y etiquetas distintos, y fuerza situaciones que el primer conjunto apenas mostraba: ramas más profundas, hojas por mayoría y casos con menos atributos disponibles. Gracias a esta prueba se comprueba que el algoritmo no depende de nombres concretos de columnas y que los textos de la ventana siguen siendo válidos con datos diferentes.

A partir de ahí se crea el panel de selección que se aprecia en la Figura 6.9. Su función es guiar al usuario antes de iniciar el entrenamiento. Primero se elige una fuente de datos, después se valida la selección y finalmente se activa el entrenamiento. La primera opción permite usar un conjunto incluido en la aplicación, mientras que la segunda prepara la carga de archivos CSV.

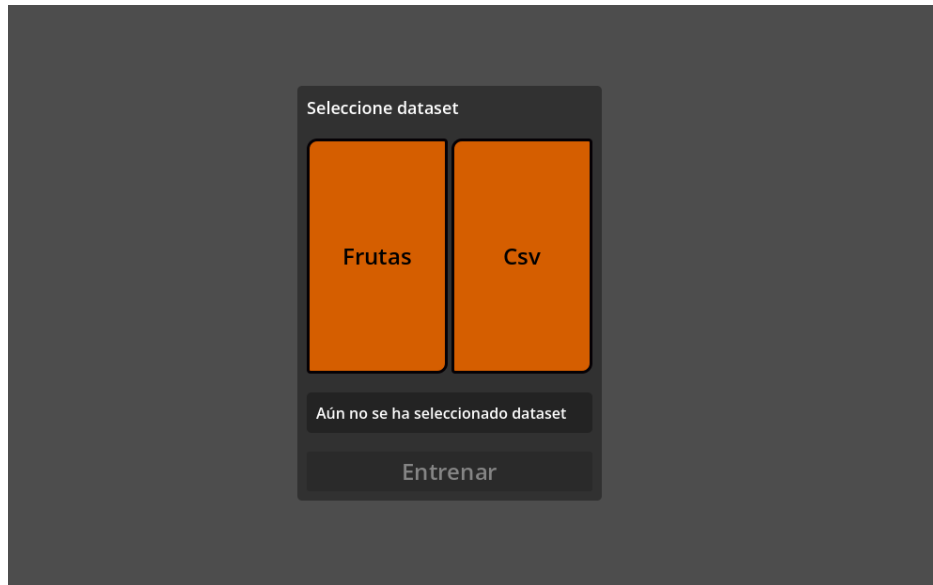


Figura 6.9: Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.

La aparición de este panel cambia el flujo del modo algorítmico. Antes, el usuario comenzaba el entrenamiento directamente. Con el panel, primero selecciona los datos, recibe un mensaje de confirmación o error y solo después puede avanzar por la construcción del árbol. La vista del árbol y la ventana informativa permanecen ocultas hasta que existe una selección válida, lo que evita mostrar una escena sin contexto.

La comunicación entre el panel de selección, el controlador y la vista se resuelve mediante Señales. Cuando el usuario confirma un conjunto de datos, el panel emite la información seleccionada y el controlador inicia el algoritmo con esos datos y atributos. Esta solución encaja con la organización de Godot, ya que permite que componentes situados en partes distintas de la Escena colaboren sin depender directamente unos de otros.

En esta etapa también se da un paso de cara a que la carga de CSVs sea más universal, al hacer posible que la columna objetivo sea una palabra distinta de *clase*. En las primeras pruebas, el algoritmo solo podía asumir ese nombre fijo para la etiqueta de salida. Con datasets distintos, esa suposición puede ser limitante. La solución adoptada consiste en tratar como atributos las columnas indicadas por el panel y deducir la etiqueta como la columna restante. Y en la carga de CSVs, se aumentó la lista de palabras que el programa reconocía automáticamente como target. **aquí he metido este anglicismo para no repetir columna objetivo** De esta forma, el árbol puede entrenarse con conjuntos que usen nombres como *class* o *label*, siempre que el conjunto tenga una única columna objetivo.

6.2.6 Carga de archivos y navegación por pasos

Tras introducir el panel, se completa la carga de CSV para que los archivos seleccionados puedan alimentar el entrenamiento. La lectura se plantea de forma práctica y limitada al tipo de datos que maneja esta versión del árbol: tablas simples, con una primera fila de cabeceras, varias filas de ejemplos y valores categóricos. La aplicación ignora filas vacías, transforma cada fila en un diccionario de valores y separa la columna objetivo del resto de atributos mediante nombres habituales o mediante la diferencia entre columnas de entrada y salida.

El panel informa al usuario del resultado de la carga. Si el archivo puede leerse y la estructura es válida, se activa el botón de entrenamiento. Si aparece un problema, se muestra un mensaje de error y se impide continuar. Esta comprobación es sencilla, aunque suficiente para evitar que el algoritmo empiece con datos incompletos o sin una etiqueta identificable.

Con la selección de datos integrada, el entrenamiento empieza a organizarse en fases. Primero el usuario escoge datos y confirma la selección, y después ve aparecer el menú del árbol que le permite avanzar para construirlo. Esta separación hace explícita la dependencia entre el conjunto de entrada y la forma final del árbol.

El siguiente cambio importante es la navegación hacia atrás. Hasta entonces, el entrenamiento solo avanzaba, lo que obligaba a reiniciar el proceso si el usuario quería revisar una decisión anterior. Para una herramienta educativa, poder retroceder es especialmente útil: permite observar de nuevo por qué se abrió una rama, por qué apareció una hoja o qué atributo se escogió en un nodo interno. Por eso se añade el botón “Paso previo”, visible en la Figura 6.10.

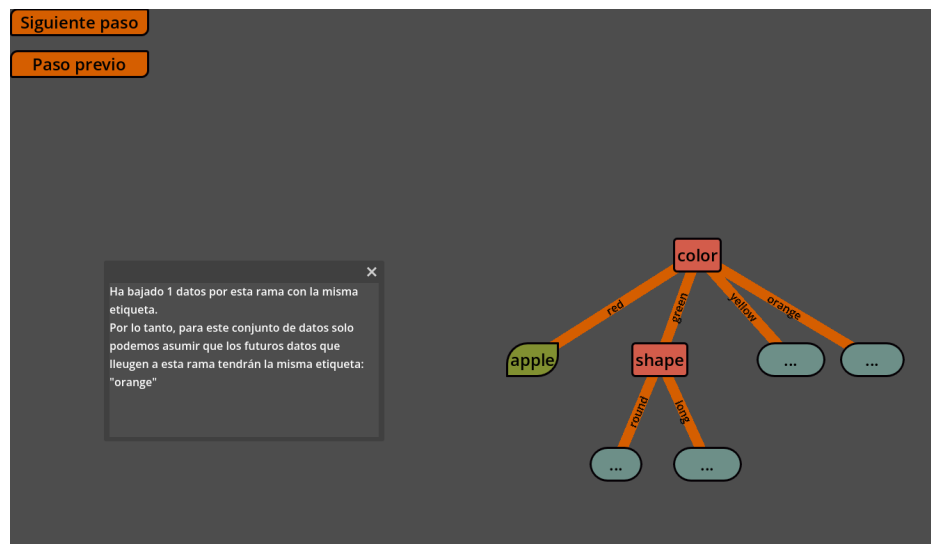


Figura 6.10: Vista del árbol con el botón para retroceder al paso anterior.

El retroceso se apoya en la misma idea que permite avanzar por pasos. El algoritmo man-

tiene una colección de contextos pendientes y otra de contextos ya ejecutados. Al avanzar, un contexto pasa de pendiente a ejecutado. Al retroceder, el último contexto ejecutado vuelve a quedar pendiente y la Escena debe deshacer su efecto visual. Esta estructura evita reconstruir todo el árbol desde cero cada vez que se pulsa “Paso previo”.

Hacer que el retroceso sea coherente requiere tratar varios casos. Si se deshace un nodo interno, también deben retirarse los contextos hijos y los nodos visuales que dependen de él. Si se deshace una rama que estaba representada por un nodo pendiente, conviene restaurar ese estado provisional en vez de eliminar toda la indicación visual. Si el usuario retrocede hasta el inicio, la raíz desaparece y el árbol debe quedar vacío. Durante las pruebas se detecta un fallo en ese último caso. Al volver al paso cero, podían quedar conexiones visibles aunque los nodos ya se hubiesen eliminado. La corrección consiste en recalcular también las conexiones cuando el árbol queda sin raíz o sin nodos. Con ello, retroceder hasta el inicio deja la vista en un estado consistente y permite avanzar de nuevo sin restos visuales de la ejecución anterior.

6.2.7 Primera evaluación visual del árbol

El último avance de esta fase es la primera versión del modo de evaluación. Hasta este punto, la aplicación enseña cómo se construye el árbol. Queda por mostrar cómo se usa ese árbol ya entrenado para clasificar nuevos ejemplos. La evaluación se plantea de forma visual: cada dato de prueba se representa como una pieza que se desplaza por la estructura siguiendo las ramas aprendidas.

Para introducir este modo se añaden nuevos controles al panel del algoritmo. Durante el entrenamiento, la evaluación permanece desactivada. Cuando el árbol termina de construirse, el botón de avanzar se bloquea y se habilita la opción de evaluar. Al activarla, se reutiliza el panel de selección de datos para escoger un conjunto de prueba. Así se distinguen dos momentos: primero se seleccionan datos para entrenar y después se seleccionan datos para comprobar el árbol resultante.

La evaluación utiliza una representación sencilla de cada ejemplo, como se aprecia en la Figura 6.11. Cada dato se dibuja como una pieza pequeña que entra en el árbol al pulsar el botón correspondiente. El recorrido se calcula leyendo el atributo de cada nodo interno y buscando la rama cuyo valor coincide con el dato evaluado. Cuando la pieza llega a una hoja, el recorrido termina y queda representada la clasificación obtenida.

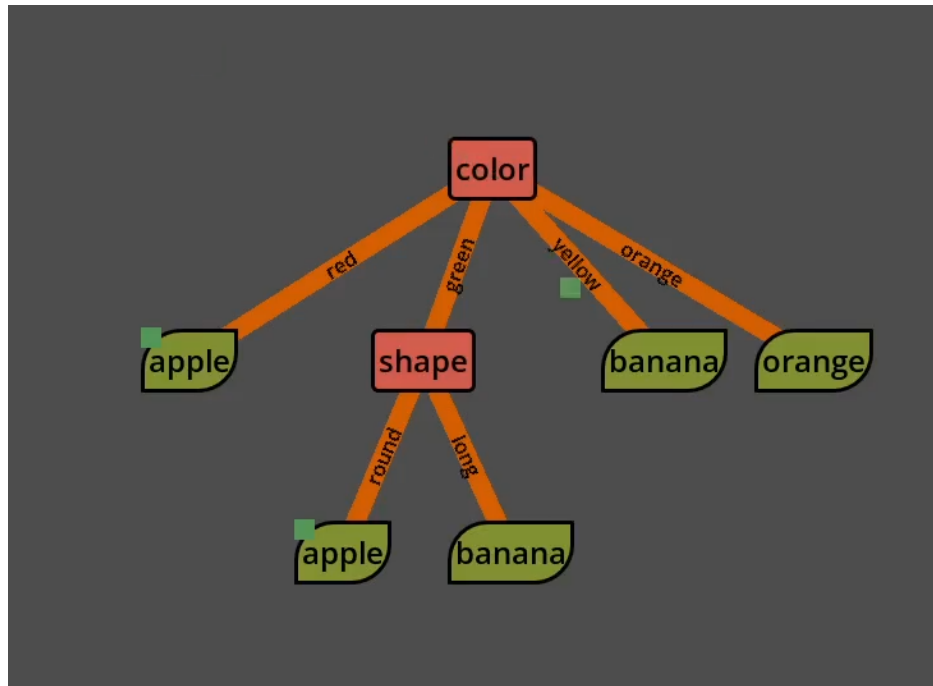


Figura 6.11: Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.

Para evitar dependencias innecesarias con el árbol en construcción, la evaluación toma una copia de la información relevante de los nodos visibles: posición, profundidad, atributo, etiqueta, valor de rama y padre. **otra lista** Con esa información puede mover los datos por la escena y decidir el siguiente nodo sin alterar la estructura entrenada. El movimiento entre nodos se anima, de forma que el usuario ve el recorrido como una secuencia de decisiones y no como un salto directo al resultado final.

Esta primera versión del modo de evaluación es todavía básica. Los datos se representan con formas simples y cada pulsación deja caer un ejemplo que recorre el árbol hasta una hoja. Quedan para fases posteriores mejoras como agrupar visualmente los ejemplos bajo las hojas, enriquecer la explicación de cada decisión o introducir una navegación más detallada del recorrido. Aun así, esta prueba establece la idea central del modo de evaluación: mostrar físicamente cómo un ejemplo atraviesa las decisiones aprendidas hasta llegar a una clasificación.

Con este conjunto de cambios queda completada la primera versión significativa del árbol de decisión. El proyecto pasa de dibujar elementos conectados a entrenar un árbol categórico paso a paso, mantener una estructura lógica sincronizada con la Escena, mostrar ramas con valores, distinguir nodos internos, hojas y nodos pendientes, animar los cambios, explicar las decisiones con texto y gráficas, cargar datos sencillos, retroceder por la construcción y evaluar ejemplos nuevos de forma visual. A partir de aquí, el desarrollo continúa con el segundo

algoritmo de la aplicación: la red neuronal.

6.3 Red neuronal: versión 0.3

Una vez completada la primera versión funcional del árbol de decisión, el proyecto incorpora el segundo modelo previsto: una red neuronal multicapa. El cambio de algoritmo también cambia el tipo de visualización necesaria. En el árbol, el entrenamiento construye una estructura que crece con nuevas ramas. En la red neuronal, la estructura se define antes de entrenar y el aprendizaje modifica valores numéricos asociados a esa estructura, principalmente pesos y sesgos. La formulación matemática del modelo, del forward pass y de la retropropagación se presenta en la Sección 5.2; aquí se describe la implementación interactiva desarrollada en Godot.

Por esta razón, el desarrollo de la red empieza por resolver un problema distinto al del árbol. Antes de mostrar el forward pass o el backward pass, hace falta que la aplicación pueda representar una arquitectura formada por capas, neuronas y conexiones densas, y que esa arquitectura tenga una correspondencia clara con el modelo lógico que se va a entrenar. La red visible debe servir como explicación y también como soporte para los cálculos posteriores: cada neurona debe poder resaltarse, cada conexión debe poder actualizarse y cada capa debe mantenerse sincronizada con los datos internos del modelo.

La versión de red neuronal sigue la misma filosofía general que ya se había usado en el árbol de decisión. La lógica matemática, la representación visual y los elementos informativos se separan en componentes distintos, comunicados mediante señales. Esta separación permite que el algoritmo avance paso a paso y que la escena se limite a mostrar lo que ocurre en cada momento. El resultado de esta fase es el núcleo funcional de la red: una arquitectura configurable, adaptada al conjunto de datos, entrenable de forma incremental y capaz de mostrar tanto el flujo de datos hacia delante como el retorno del error.

6.3.1 Primera arquitectura visual de la red

El primer paso consiste en añadir una vista propia para redes neuronales y conectarla con el menú principal. Hasta ese momento, la aplicación estaba organizada alrededor del árbol de decisión. La nueva vista introduce una Escena independiente, con sus propios nodos de interfaz y sus propios objetos visuales, de manera que el desarrollo del segundo algoritmo no quede mezclado con la Escena del árbol.

La representación inicial de la red se construye a partir de tres elementos básicos: capas, neuronas y conexiones. Las capas se colocan en horizontal para mostrar el recorrido natural de los datos, desde la entrada situada a la izquierda hasta la salida situada a la derecha. Dentro de cada capa aparecen las neuronas, representadas como elementos interactivos. Esta elección

permite que más adelante puedan resaltarse durante el entrenamiento o responder cuando el usuario las pulse para consultar información.

La red parte de una estructura mínima formada por una capa de entrada y una capa de salida. Esta decisión proporciona una base visual sencilla sobre la que ir añadiendo complejidad. Las capas ocultas se incorporan después, cuando el menú de creación permite configurar la arquitectura. Desde esta primera fase, la vista se piensa como una estructura dinámica: el número de capas y neuronas puede cambiar, y la Escena debe reconstruir las conexiones para seguir representando una red coherente.

Las conexiones se dibujan entre capas consecutivas y siguen una estructura densa. Esto significa que cada neurona de una capa queda conectada con todas las neuronas de la capa siguiente. En una primera lectura, estas líneas sirven para mostrar la topología de la red. Más adelante pasan a representar también información del modelo, ya que su color y grosor dependen del peso asociado. [forshadowin](#) Con ello, la Escena deja preparado el vínculo entre la arquitectura visible y los valores numéricos que el entrenamiento modificará. En la Figura 6.12 se muestra esta primera representación visual.



Figura 6.12: Primera arquitectura visual de la red neuronal, formada por capas, neuronas y conexiones densas entre capas consecutivas.

Esta primera arquitectura también fija una separación importante entre la red que se ve y la red que se calcula. La Escena de Godot se encarga de colocar capas, neuronas y conexiones, mientras que el estado lógico conserva la estructura y los valores que definen el modelo. Mantener ambas partes separadas evita que la visualización se convierta en la única fuente

de información de la red. La vista puede reconstruirse, animarse o actualizar sus conexiones sin perder la referencia al modelo lógico que después usará el algoritmo de entrenamiento.

6.3.2 Configuración de la arquitectura y representación lógica

Una vez definida la escena básica de la red, el siguiente paso consiste en permitir que el usuario configure su arquitectura desde la propia aplicación. Para ello se crea un menú de construcción que controla el número de neuronas de entrada, el número de neuronas de salida, la cantidad de capas ocultas, las neuronas de cada capa oculta y las funciones de activación asociadas. Con este panel, la arquitectura puede editarse dentro de unos límites conocidos. La selección de funciones de activación dentro de este menú se aprecia en la Figura 6.13.

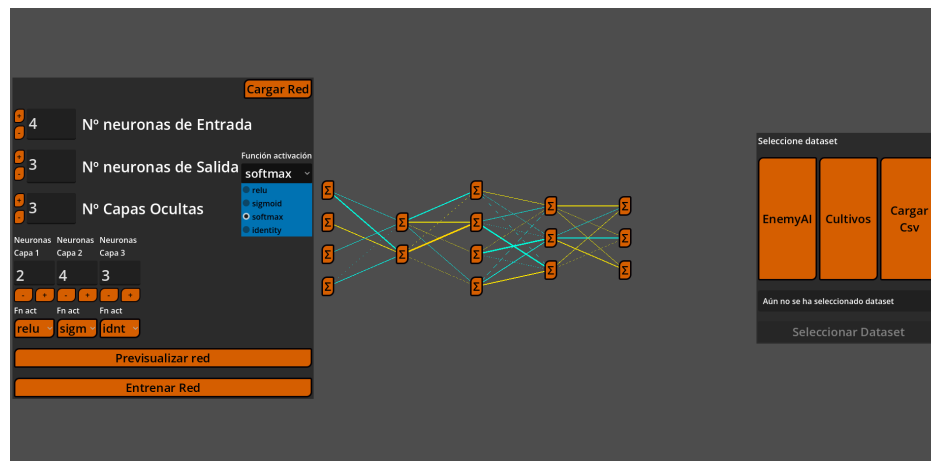


Figura 6.13: Menú de creación de la red neuronal con selectores para las funciones de activación de las capas.

Estos límites son necesarios porque la aplicación tiene una finalidad didáctica y debe seguir siendo manejable. Por esto se establecen máximos para el número de capas y para el número de neuronas por capa, de forma que la visualización no crezca hasta ocupar un espacio imposible de seguir. También se validan los valores introducidos en el menú antes de aplicarlos, para evitar una cantidad negativa de neuronas o una capa vacía.

En cuanto a la representación lógica de la red, esta se organiza mediante una convención simple de identificadores. La capa de entrada usa el identificador 0, las capas ocultas usan enteros positivos y la capa de salida usa el identificador -1. Esta decisión permite tratar la salida como un elemento especial y, al mismo tiempo, recorrer las capas ocultas en orden natural. También facilita que las señales de Godot indiquen con claridad qué capa debe crear neuronas, eliminarlas, cambiar su activación o actualizar sus conexiones. Pero el modelo lógico guarda algo más que el número de capas. También conserva las matrices de pesos que conectan cada capa con la anterior y la función de activación que debe aplicarse en cada una. Cada matriz se

entiende desde la capa de destino: cada fila representa una neurona de esa capa y contiene los pesos que llegan desde todas las neuronas de la capa previa. Esta forma encaja con el cálculo posterior de una neurona, que toma sus entradas, las multiplica por sus pesos y obtiene un valor antes de aplicar la activación.

La capa de entrada recibe un tratamiento distinto en este contexto. Su función es mostrar los valores que entran en la red, por lo que no necesita una transformación aprendida como las capas posteriores. Esto se traduce en que sus pesos son todos 1, y no varían, y su función de activación es la lineal. En la estructura lógica se mantiene como parte de la arquitectura para que la vista pueda dibujar sus neuronas y conectarlas. Pero la parte entrenable empieza a partir de las capas que reciben esos valores.

si hay que borrar, esto podría sobrar Un detalle importante es que, durante la edición de la red, se diferencia entre el estado temporal y el estado definitivo de la red. El menú modifica una propuesta de arquitectura almacenada en diccionarios temporales, y esa propuesta se copia al modelo principal cuando la red se recarga o cuando comienza el entrenamiento. Esta separación evita que cada cambio parcial del formulario se convierta inmediatamente en la red que va a entrenarse, ayudando a disminuir el número de actualizaciones. Esto se hizo como medida preventiva para asegurar la fluidez durante el proceso de creación de la red.

La modificación de una capa obliga a actualizar otras partes de la red lógica. Si cambia el número de neuronas de una capa, también cambia el número de pesos que necesita la capa siguiente, ya que cada una de sus neuronas debe recibir una conexión desde todas las neuronas anteriores. Si se añade una capa oculta, se crean sus pesos iniciales y se reajusta la capa de salida para conectarse con la nueva capa previa. Si se elimina una capa, se borran sus datos y se vuelve a enlazar la salida con la última capa que permanece en la arquitectura. De esta forma, el modelo conserva siempre una topología densa y consistente.

Y la representación visual debe seguir esos mismos cambios. Al añadir o eliminar capas y neuronas, el contenedor de conexiones densas recalcula los enlaces entre capas consecutivas y borra los que ya no son válidos. Esta corrección resulta importante cuando se insertan capas ocultas, porque podrían quedar conexiones antiguas entre entrada y salida, ajenas a la red real. Las actualizaciones se realizan de forma diferida cuando es necesario, para que Godot termine de crear o eliminar nodos antes de que las líneas intenten consultar sus posiciones. Así nunca se rompe el orden visual de la red.

Con esta organización, la arquitectura editable queda conectada con una representación lógica estable. El menú actúa como interfaz de configuración, la escena traduce esa configuración a capas, neuronas y conexiones, y el modelo interno conserva los pesos y activaciones que después usará el algoritmo. Este paso deja preparada la red para una condición que se resuelve a continuación: adaptar automáticamente la arquitectura al conjunto de datos elegido.

6.3.3 Selección de datos y restricciones de la red

La configuración manual de toda la arquitectura solo tiene sentido si se escogen de forma arbitraria las entradas y salidas. Pero en la realidad, las capas de entrada y salida están restringidas por el conjunto de datos. Cada atributo usado como entrada necesita una neurona en la primera capa, y cada clase o variable objetivo necesita una representación en la salida. Por este motivo se incorpora un panel de selección de datos específico para la red neuronal. El panel ofrece datasets internos y carga de ficheros CSV. Los datasets internos sirven para cubrir los dos tipos de problema principales que soporta la aplicación: clasificación y regresión. Esta diferencia es relevante porque la red que se crea para cada caso tiene una capa final cuyo número de neuronas se decide de forma distinta.

Ahora cuando el usuario carga un CSV, la aplicación muestra las columnas disponibles y permite seleccionar explícitamente una o varias como objetivo. Esta decisión hace que la carga de datos sea más flexible que en las primeras pruebas, donde se dependía de nombres habituales como *class*, *label* o *target*. El resto de columnas se tratan como atributos de entrada. En la Figura 6.14 se ve este selector de columnas objetivo.

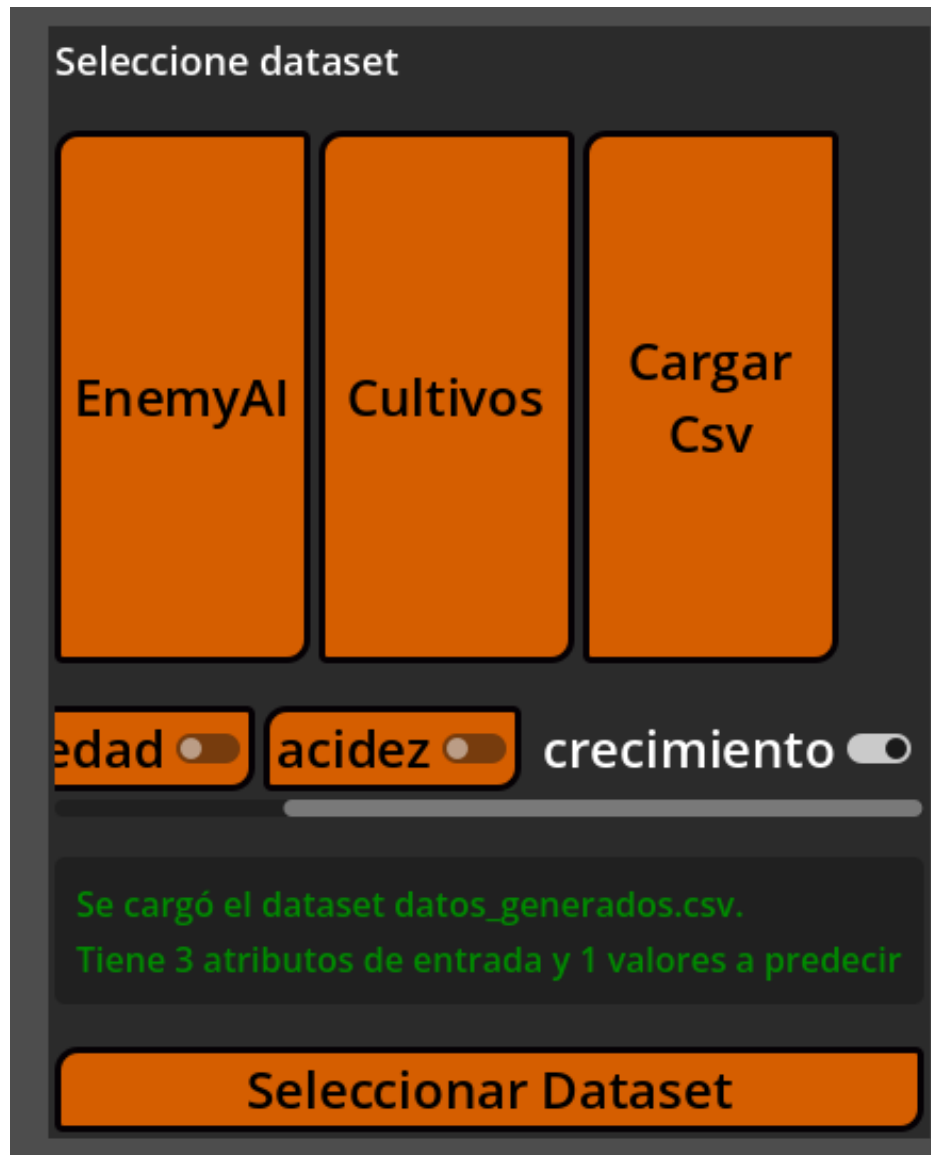


Figura 6.14: Selección de columnas objetivo para la red neuronal a partir de un dataset cargado desde un archivo CSV.

El panel analiza los tipos de las columnas seleccionadas para decidir cómo preparar el problema. Si hay una única variable objetivo categórica, o una única variable entera tratada como clase, el problema se considera de clasificación. Si las variables objetivo son numéricas continuas, el problema se considera de regresión. Los valores numéricos se normalizan cuando es necesario y las entradas categóricas se convierten a valores enteros para que puedan circular por la red. En clasificación aparece otra transformación importante. Las etiquetas textuales, y también las clases enteras que no forman una secuencia compacta, se codifican en identificadores contiguos. Así, una clase con valor original 10 queda asignada a un identificador

interno compacto. La red solo necesita una neurona por clase real, y la aplicación conserva un decodificador para volver a mostrar los nombres originales en la interfaz. Todos estos cambios sirven para procesar de forma sencilla el dataset para que se ajuste a las necesidades de entrada y salida de una red neuronal.

Con la información del dataset se construye un conjunto de restricciones para el menú de creación. La capa de entrada queda fijada al número de atributos seleccionados. La capa de salida queda fijada al número de clases o variables objetivo. La función de activación de salida también queda determinada por el tipo de problema: softmax para clasificación e identidad para regresión. Cuando esta restricción llega al menú, el selector de activación final se ajusta y queda bloqueado para evitar una configuración incoherente.

Mientras, el resto de la arquitectura sigue siendo configurable. El usuario puede modificar capas ocultas, número de neuronas internas y funciones de activación intermedias, siempre dentro de los límites definidos en el menú. De esta manera, la aplicación combina libertad de experimentación con las restricciones mínimas que impone el dataset. La red creada siempre recibe vectores del tamaño correcto y produce salidas que pueden compararse con los valores esperados.

La selección de datos también mejora la lectura de la red visible. Las neuronas de entrada pasan a mostrar los nombres de los atributos, y las neuronas de salida muestran las clases decodificadas o las variables objetivo de regresión. Con estos nombres, cada extremo del modelo representa información reconocible del dataset y la lectura de la arquitectura resulta más concreta. Esto es muy relevante de cara a que el usuario relacione lo que se le muestra con lo que está sucediendo. Esta relación entre dataset y red visible se muestra en la Figura 6.15, donde las neuronas aparecen etiquetadas con información del conjunto seleccionado.

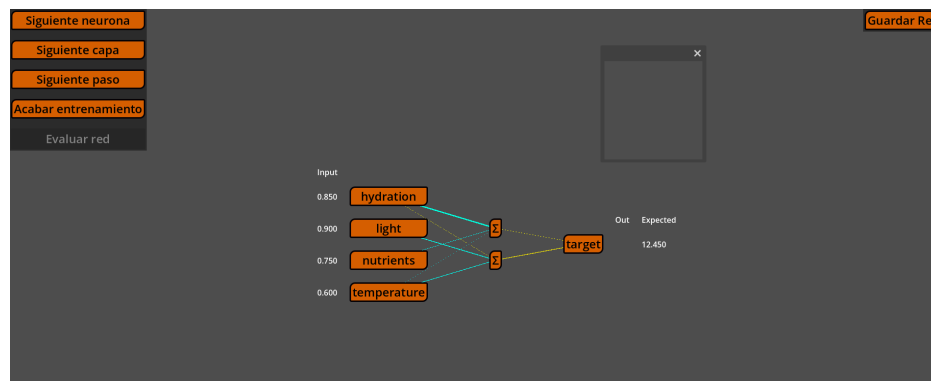


Figura 6.15: Red neuronal durante el entrenamiento, con nombres procedentes del dataset en las neuronas de entrada y salida, y con las columnas visuales de entrada, salida y valor esperado.

Con esta dependencia entre datos y arquitectura, el flujo de la vista se reorganiza. Primero

se selecciona el dataset y después se crea la red. Este orden evita que el usuario configure una arquitectura incompatible y, al mismo tiempo, permite que el menú aparezca ya inicializado con las entradas, salidas y activación final adecuadas. Una vez fijadas estas condiciones, la red queda preparada para conectarse con el algoritmo de entrenamiento incremental.

6.3.4 Algoritmo de entrenamiento incremental

Una vez que la arquitectura y los datos están definidos, la red necesita una pieza encargada de ejecutar el entrenamiento. Esta responsabilidad se concentra en un Nodo de algoritmo, separado de la vista. Su papel es equivalente al que cumplía el algoritmo del árbol: calcula el siguiente avance, actualiza el estado interno y emite señales para que la Escena represente el cambio correspondiente. La vista, por tanto, puede centrarse en representar el avance recibido mediante señales.

Al comenzar el entrenamiento, la Escena configura el algoritmo con toda la información necesaria. Le entrega las filas del dataset, los atributos de entrada, las columnas objetivo, y toda la información de la arquitectura de la red. Con esa información, el algoritmo prepara el orden de capas, reinicia sus cursores internos y deja lista la primera muestra. El mismo mecanismo sirve más adelante para inferencia, con una configuración que recorre la red sin actualizar pesos.

Una vez iniciado, entrenamiento se organiza como una máquina de estados. Primero se carga una muestra del dataset y se colocan sus valores de entrada en la capa inicial. Después se ejecuta el forward pass, se compara la salida calculada con el objetivo esperado, se prepara el retorno visual del error y se ejecuta el backward pass. Al terminar una muestra, el algoritmo avanza a la siguiente usando los pesos que acaban de actualizarse. Este recorrido se repite hasta completar los datos previstos o hasta que se alcanza el estado final. Esta secuencia sigue el proceso formal descrito en la Sección 5.2, adaptado aquí a una ejecución observable paso a paso.

Para sostener este proceso, el algoritmo conserva varios diccionarios internos. Uno almacena las salidas activadas de cada capa, otro conserva las sumas ponderadas previas a la activación y otro guarda los deltas calculados durante la retropropagación. Estos valores tienen una doble utilidad: permiten continuar el cálculo sin repetir operaciones ya realizadas y sirven como fuente para las explicaciones mostradas en la ventana informativa.

El proceso de aprendizaje se controla desde un panel específico. El usuario puede avanzar una sola neurona, una capa completa, una muestra completa o terminar todo el entrenamiento pulsando diferentes botones. Cada opción llama al mismo proceso interno, cambiando únicamente hasta dónde debe avanzar antes de devolver el control a la interfaz. Así se mantiene una única lógica de entrenamiento y se ofrecen varios niveles de detalle para observar el proceso. La Figura 6.16 muestra este panel de control.

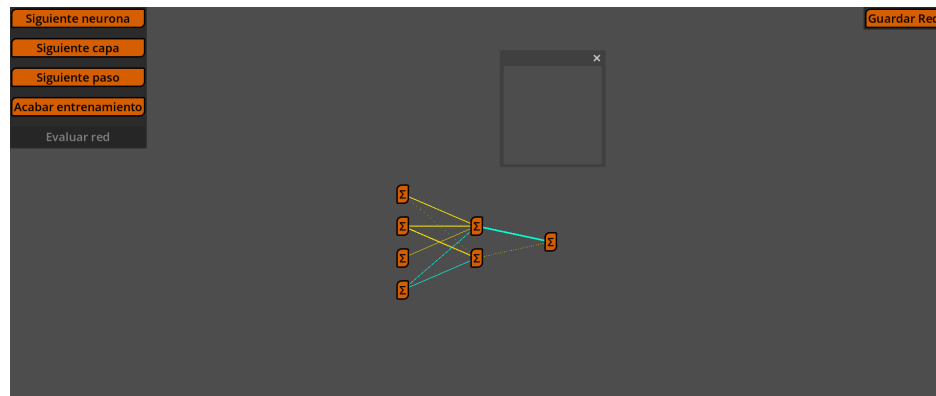


Figura 6.16: Panel de entrenamiento de la red neuronal con controles para avanzar con distintos niveles de granularidad.

La ejecución por muestras resulta importante para que el aprendizaje sea visible. Cada fila del dataset produce una entrada concreta, una salida concreta, un error concreto y una actualización de pesos concreta. La siguiente fila ya utiliza la red modificada por la anterior, de modo que el usuario puede entender el entrenamiento como una sucesión de pequeñas correcciones sobre el mismo modelo. En la realidad las redes neuronales usan mecanismos algo más complejos para evitar saltos bruscos en los valores de los pesos, pero aquí se decidió realizar la actualización directamente para simplificar el proceso y hacerlo más directo, y por ende entendible.

Además algoritmo también coordina los elementos visuales que acompañan al entrenamiento. Al cargar una muestra, emite Señales para preparar los valores de entrada y los valores esperados. Durante el forward pass y el backward pass, emite Señales de neurona y de capa para resaltar la parte de la red que se está procesando. Cuando un peso cambia, emite la actualización correspondiente para que la conexión visual modifique su grosor o color según el nuevo valor.

Al finalizar el entrenamiento, la interfaz cambia de estado. Los botones de avance quedan desactivados y se habilita la evaluación de la red. Este cierre evita que el usuario continúe modificando una ejecución ya terminada y marca el paso desde el aprendizaje hacia el uso del modelo entrenado.

6.3.5 Forward pass y visualización de datos

El forward pass es la fase en la que una muestra atraviesa la red desde la capa de entrada hasta la capa de salida. En la aplicación se ejecuta capa a capa y, dentro de cada capa, neurona a neurona. Esta granularidad permite que el usuario avance con mucho detalle y vea qué parte de la red participa en cada instante.

Al cargar una muestra, el algoritmo extrae sus valores de entrada siguiendo el orden de

los atributos seleccionados. Ese vector se guarda como salida de la capa de entrada, ya que la primera capa entrenable usa esos valores como datos de partida. En paralelo, la escena muestra esos valores junto a las neuronas de entrada mediante pequeños elementos visuales asociados a cada atributo. Al comenzar el cálculo, los valores se animan hacia la red y desaparecen, reforzando la idea de que la muestra entra en el modelo.

Para cada neurona, el cálculo sigue la operación de una capa densa explicada en la Sección 5.2. El algoritmo combina los valores de la capa anterior con los pesos de entrada de la neurona, incorpora su sesgo y aplica la función de activación configurada para la capa. Así, cada avance produce un valor intermedio que puede mostrarse y usarse como entrada de la capa siguiente.

La activación softmax requiere un tratamiento específico dentro de la implementación. Como sus salidas dependen del vector completo de la capa, primero se calculan todos los valores previos a la activación y después se aplica la función al conjunto completo. La explicación de la ventana informativa refleja este comportamiento para presentar softmax como una operación de capa.

Durante el forward pass se guardan dos tipos de valores por capa: las sumas ponderadas previas a la activación y las salidas activadas. Esta caché permite que la explicación, el cálculo del error y el backward pass usen exactamente los mismos resultados que se obtuvieron al propagar la muestra hacia delante.

La salida de la red también se representa visualmente. Junto a la capa final aparece una columna de valores producidos por la red, y junto a ella aparece otra columna con el valor esperado. Cuando se calcula una neurona de salida, su valor se añade a la columna de salida mientras los valores anteriores de la misma muestra se mantienen visibles. De esta manera, la predicción se completa gradualmente y queda alineada con las clases o variables objetivo que se habían mostrado en la arquitectura, como se veía en la Figura 6.15.

El resaltado de neuronas y capas acompaña todo el proceso, haciendo que cada vez que se evalúa una neurona, esta cambie de estado visual y la ventana informativa reciba los datos necesarios para explicar el cálculo. Al terminar una capa, el vector de salidas activadas pasa a ser la entrada de la capa siguiente. El usuario puede seguir así el recorrido completo de la muestra, desde los atributos iniciales hasta la predicción final.

Cuando termina la capa de salida, el flujo se bifurca según el modo de ejecución. En entrenamiento, el algoritmo pasa a comparar la salida con el objetivo esperado. En inferencia, la muestra termina en ese punto, porque evaluar una red entrenada consiste únicamente en propagar los datos hacia delante.

6.3.6 Cálculo del error y backward pass

Tras completar el forward pass, la red ya dispone de una salida para la muestra actual. El siguiente paso consiste en compararla con el objetivo esperado y convertir esa diferencia en una señal de error que pueda volver por la red. La interfaz representa esta transición desplazando la columna de salida y transformando la columna de valores esperados en valores de error o delta. Así se marca visualmente el cambio entre producir una predicción y corregir el modelo a partir de ella.

El backward pass recorre las capas en orden inverso, empezando en la capa de salida, donde se conoce la diferencia entre lo calculado y lo esperado, y continuando hacia las capas ocultas. En cada neurona se calcula un delta, que resume cómo debe cambiar esa neurona para reducir el error de la muestra. Ese delta se usa después para actualizar los pesos que llegan a la neurona.

En la capa de salida hay varios casos, siguiendo las expresiones recogidas en la Sección 5.2. Cuando el problema es de clasificación y se usa softmax con entropía cruzada, el delta coincide con la diferencia vectorial entre la salida activada de la red y el vector esperado. Para otras combinaciones de pérdida y activación, el algoritmo calcula primero el gradiente de la pérdida respecto a la salida activada y lo combina con la derivada local de la función de activación. Con esto, la salida genera el primer conjunto de deltas que iniciará la retropropagación.

Las capas ocultas reciben el error de la capa siguiente. Para cada neurona oculta se calcula cuánto ha contribuido a los deltas posteriores, ponderando esos deltas con los pesos de salida de la neurona. Después se tiene en cuenta la derivada de su propia activación. La idea que se muestra al usuario es que una neurona oculta se corrige según su influencia en las neuronas que vienen después.

Una vez calculado el delta de una neurona, se actualizan sus pesos y su sesgo mediante descenso de gradiente. Cada peso se corrige en función del delta de la neurona, del valor que llega desde la capa anterior y de la tasa de aprendizaje configurada. El sesgo se actualiza con el propio delta, de forma coherente con la formulación de la Sección 5.2. Su inclusión fue importante para que la red pudiera aprender desplazamientos y relaciones que no pasan necesariamente por el origen. Las pruebas externas con scripts de Python y una red equivalente en PyTorch ayudaron a comprobar este comportamiento en datasets sintéticos sencillos.

Cada actualización se sincroniza con la representación lógica de la red. Los pesos y sesgos modificados se copian de vuelta al modelo compartido, y cada peso emite una señal de actualización. El contenedor de conexiones recibe esa señal y redibuja la línea correspondiente con el color y grosor derivados del nuevo valor. De esta manera, el backward pass se observa como un cambio visual del modelo mientras aprende.

Al completar la última capa oculta, la muestra queda terminada. El algoritmo guarda la información necesaria para explicar el paso, prepara la siguiente fila del dataset y repite el

proceso con los pesos ya actualizados. El entrenamiento queda así estructurado como una secuencia de predicciones, errores y correcciones acumuladas sobre la misma red.

6.3.7 Ventana informativa de la red neuronal

La red neuronal necesita una ventana informativa más rica que la del árbol, porque el aprendizaje depende de operaciones numéricas encadenadas. Ver una neurona resaltada o una conexión cambiar de grosor ayuda a seguir el proceso. La ventana completa esa representación al explicar qué cálculo acaba de producir cada cambio. Para ello, recibe datos internos del algoritmo y los convierte en texto y fórmulas comprensibles.

La ventana se mantiene separada del algoritmo y de la Escena. El algoritmo emite señales con la información del paso actual, y la ventana decide cómo presentarla. Esta organización permite que el cálculo siga estando en la clase de entrenamiento, que la red visible se centre en resaltar neuronas y conexiones, y que la explicación textual evolucione sin mezclar responsabilidades.

Una primera utilidad implementada de la ventana aparece al pulsar una neurona. Al hacerlo se muestran los pesos que llegan a esa neurona y su sesgo asociado. Esta consulta manual permite inspeccionar el estado del modelo durante el entrenamiento, revisando los valores de las conexiones representadas. Si el usuario deselectiona la neurona, la ventana recupera la explicación anterior para no perder el contexto del paso que se estaba visualizando.

Durante el forward pass, la ventana explica el cálculo de la neurona resaltada. Muestra las entradas recibidas, los pesos utilizados, el sesgo, la suma ponderada y la salida tras aplicar la función de activación. La explicación sigue la misma estructura matemática que usa el algoritmo y remite a la formulación general de la Sección 5.2. En el caso de una neurona concreta, la ventana desarrolla esa operación con los valores de entrada y pesos correspondientes.

La ventana también puede explicar una capa completa. Cuando el avance se realiza por capas, la explicación pasa de una operación escalar a una operación matricial. Esto ayuda a relacionar el cálculo neurona a neurona con la formulación habitual de una red neuronal, donde todas las salidas de una capa pueden obtenerse a partir de un vector de entrada, una matriz de pesos y un vector de sesgos. Si la activación es softmax, el texto aclara que la función se aplica al vector completo de la capa.

En la fase de error, la ventana muestra la comparación entre salida y objetivo esperado. Indica el tipo de pérdida usado, el valor de la pérdida y los deltas que iniciarán la retropropagación. Esta explicación actúa como puente entre el forward pass y el backward pass: primero se ve qué ha producido la red, después se explica cómo ese resultado se transforma en una señal de corrección.

Durante el backward pass, la ventana adapta el texto al tipo de capa que se está procesando. En la salida puede mostrar la simplificación de softmax con entropía cruzada, o la combinación

de derivada de pérdida y derivada de activación en otros casos. En las capas ocultas explica el gradiente entrante como la suma de contribuciones procedentes de la capa siguiente. Después muestra cómo ese delta genera gradientes de pesos y cómo esos gradientes modifican los parámetros.

La granularidad de la explicación cambia con el modo de avance elegido por el usuario. Si se avanza neurona a neurona, la ventana se centra en una operación pequeña. Si se avanza por capas, resume el cálculo de toda la capa. Si se avanza una muestra completa, la ventana puede resumir el dato procesado, su salida, el error y la actualización realizada. Así, la misma infraestructura permite observar el entrenamiento con distintos niveles de detalle.

Las fórmulas pueden mostrarse como texto plano o con un renderizado matemático más elaborado. Ese soporte pertenece en buena medida a las mejoras de interfaz posteriores, por lo que aquí lo relevante es su papel didáctico: acompañar cada paso con una representación simbólica del cálculo. Con la ventana informativa, la red combina la animación de valores con el razonamiento matemático que justifica cada cambio del modelo.

6.3.8 Inferencia con la red entrenada

Al terminar el entrenamiento, la aplicación habilita la evaluación de la red. Esta fase permite comprobar cómo se comporta el modelo entrenado con datos compatibles, usando el mismo tipo de visualización que se había preparado durante el aprendizaje. El cambio de modo es importante porque la red pasa de ajustar sus parámetros a aplicar los parámetros que ya ha aprendido.

La inferencia reutiliza el recorrido del forward pass. Cada muestra se carga, sus valores aparecen junto a la capa de entrada y después se propagan por las capas hasta producir una salida. La diferencia principal está en que el algoritmo se configura con tasa de aprendizaje nula y con el modo de inferencia activado. Cuando se completa la capa de salida, la muestra se da por finalizada: no se calcula retorno del error ni se actualizan pesos o sesgos.

Antes de ejecutar la inferencia, la aplicación valida el dataset elegido para evaluar. Debe tener los mismos atributos de entrada que el dataset usado en entrenamiento, las mismas salidas esperadas y el mismo orden de columnas. Esta comprobación evita alimentar la red con vectores que tengan una estructura distinta a la que aprendió. Sin esa validación, una neurona de entrada podría recibir un atributo diferente al que representa su peso entrenado.

La visualización de datos se aprovecha también en este modo. Los valores de entrada vuelven a situarse junto a sus neuronas, se animan hacia la red y la columna de salida se completa a medida que avanza el cálculo. Al no haber backward pass, la evaluación se entiende como una lectura directa del modelo: se introduce una muestra y se observa qué salida produce.

Con esta fase queda completo el núcleo funcional de la red neuronal dentro de la aplicación. La herramienta permite escoger datos, crear una arquitectura coherente con ellos,

entrenarla paso a paso, mostrar el forward pass, calcular y explicar el backward pass, actualizar los pesos de forma visible y evaluar la red entrenada mediante inferencia. Las mejoras posteriores se centran en pulir la interfaz general, adaptar mejor la vista a distintos tamaños y mejorar la presentación visual, partiendo ya tanto de un árbol como de una red funcionales y explicables.

6.4 Mejoras de funcionalidades e interfaz de usuario

Una vez completados los núcleos funcionales del árbol de decisión y la red neuronal, el desarrollo cambia de enfoque. Los dos algoritmos principales ya están presentes en la aplicación: el árbol de decisión puede construirse, explicarse y evaluarse, y la red neuronal puede configurarse, entrenarse e inspeccionarse paso a paso. A partir de este punto, el trabajo se concentra en convertir ese conjunto de Escenas funcionales en una herramienta más cómoda, coherente, preparada y útil para su uso desde la versión web.

Esta fase se plantea como una revisión de tareas transversales que afectan a la experiencia completa, una vez terminada la incorporación de los algoritmos principales. Algunas mejoras afectan a toda la aplicación, como la navegación, el diseño, la adaptación a web o la limpieza del estado al cambiar de Escena. Otras se centran en reforzar partes concretas, como la evaluación del árbol o las explicaciones de la red neuronal. En conjunto, estas tareas permiten pasar de una demostración técnica de los modelos a una aplicación que cumple el objetivo de ayudar a comprender los algoritmos propuestos.

6.4.1 Entrada principal y navegación común

Una de las primeras tareas de pulido consiste en revisar el menú principal. Hasta ese momento, la pantalla inicial funcionaba solo como una forma rápida de entrar en cada algoritmo. Con la aplicación más avanzada, el menú necesitaba presentar mejor el contenido disponible y guiar al usuario para que comprenda mejor qué hay en pantalla. Para ello se sustituye que los botones den acceso directo a los algoritmos por un flujo de selección. El usuario escoge primero el algoritmo que quiere visualizar y, al hacerlo, la interfaz actualiza un panel descriptivo con información sobre esa opción. El botón de comienzo se habilita solo cuando existe una selección válida. De esta forma el menú deja de ser una lista de botones aislados y pasa a actuar como punto de entrada explicativo al proyecto.

Además de explicaciones, también se añaden previsualizaciones de los algoritmos. La selección del árbol o de la red va acompañada de una imagen representativa, lo que ayuda a anticipar el tipo de escena que se abrirá a continuación. Esta decisión mantiene la filosofía visual de la aplicación: siempre que sea posible, la interfaz combina texto con una referencia gráfica que facilite la orientación.

Para planificar esta reorganización se preparó un boceto previo con Balsamiq Wireframes [15], una herramienta de creación de wireframes de baja fidelidad que permite definir la estructura de una interfaz antes de implementarla. El mockup de la Figura 6.17 sirvió como guía para ordenar la pantalla inicial alrededor de una selección de algoritmo, un panel descriptivo y una acción principal de comienzo. Durante la implementación final se ajustaron textos, tamaños e imágenes, por lo que la versión terminada que se muestra en las Figuras 6.18 y 6.19 conserva la idea general del boceto con algunos cambios visuales.

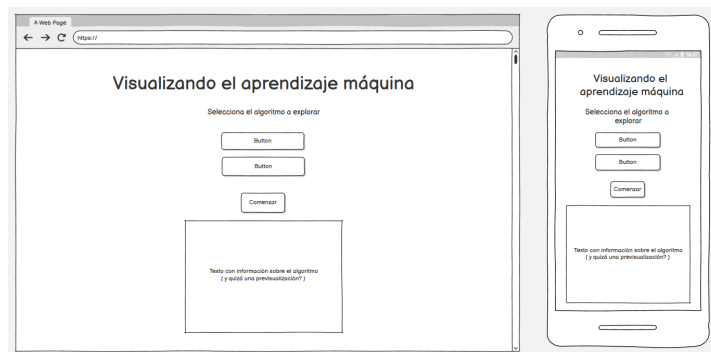


Figura 6.17: Mockup del menú principal realizado con Balsamiq antes de implementar la reorganización final de la entrada a la aplicación.

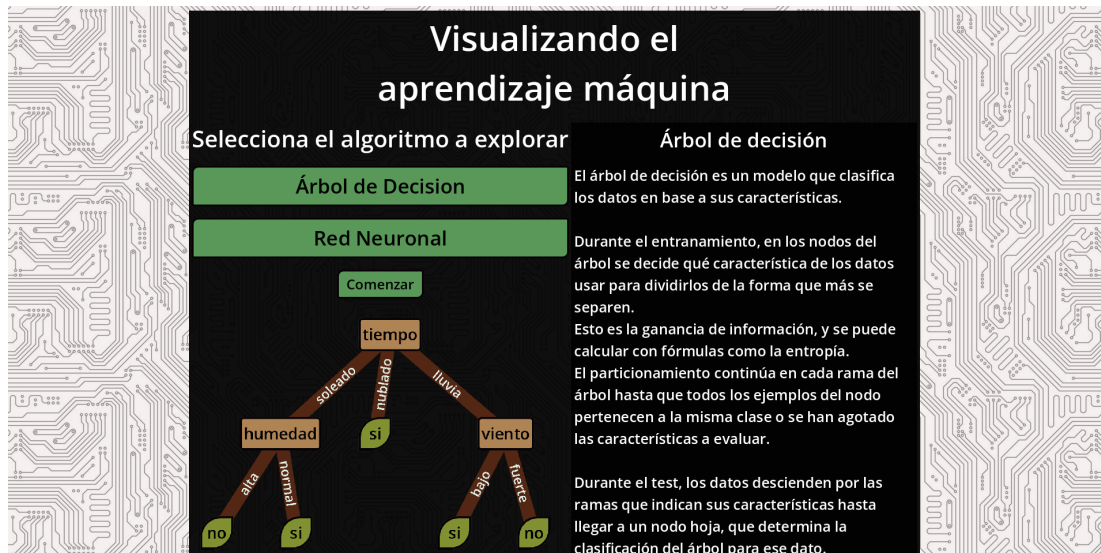


Figura 6.18: Versión final del menú principal, mostrando seleccionado el árbol de decisión.

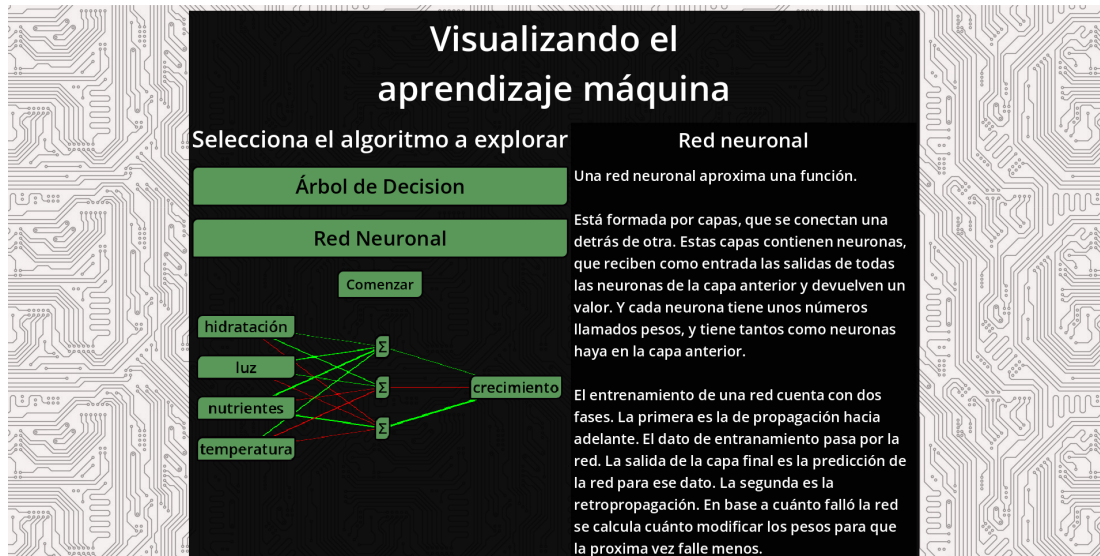


Figura 6.19: Versión final del menú principal, mostrando seleccionada la red neuronal

aquí sería donde se contaría lo de el menú de configuración, si se hace

La navegación interna se unifica mediante un menú común que se aprecia en la Figura 6.20. Tanto el árbol como la red incorporan una opción para volver al menú principal y, cuando corresponde, botones asociados a cargar o guardar el algoritmo que se esté explorando. Este menú es una Escena a parte con toda la lógica autocontenida, de modo que las vistas no tienen que resolver por separado la navegación básica. En el árbol, los botones de carga y guardado se relacionan con la persistencia de la estructura entrenada. En la red, el guardado abre el panel de exportación del modelo. Al volver al menú principal también se limpia el estado compartido de la red neuronal. Sin esto, volver a intentar crear una red no la construía bien por interferencias con la red anterior. esto es una explicación de un bug, pero ns, no hay más q decir Esta mejora en la navegación tiene un efecto importante sobre la experiencia de uso, ya que ahora no hay que salir de la misma reiniciando la página web para continuar experimentando con ella.

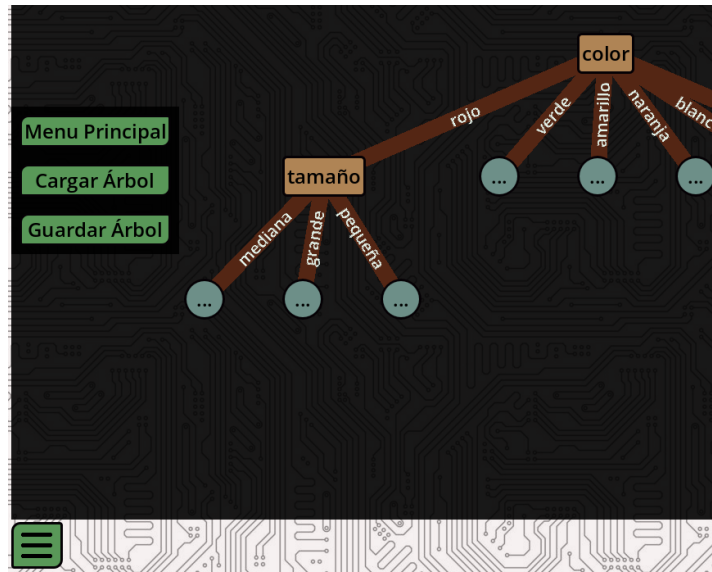


Figura 6.20: Menu de navegacion que permite volver al menú principal y guardar y cargar estructuras de los algoritmos.

6.4.2 Adaptación a web, móvil y pantallas de distinto tamaño

Otra tarea central de esta fase es la adaptación de la interfaz a tamaños de pantalla distintos. La aplicación se publica como página web, por lo que no puede asumir una única resolución de escritorio. El mismo contenido debe seguir siendo utilizable en una ventana amplia, en una pantalla vertical y en dispositivos con interacción táctil.

Antes de llevar esta adaptación a Godot, se definieron con Balsamiq dos disposiciones base para las vistas de los algoritmos. Los bocetos de la Figura 6.21 ayudan a fijar la diferencia entre la versión horizontal, pensada para colocar la visualización y la ventana informativa en paralelo, y la versión vertical, pensada para apilar los mismos elementos en pantallas estrechas. Esta planificación reduce la dependencia de posiciones absolutas y permite tratar la adaptación como una decisión de estructura desde el inicio, más allá de un ajuste de escala.

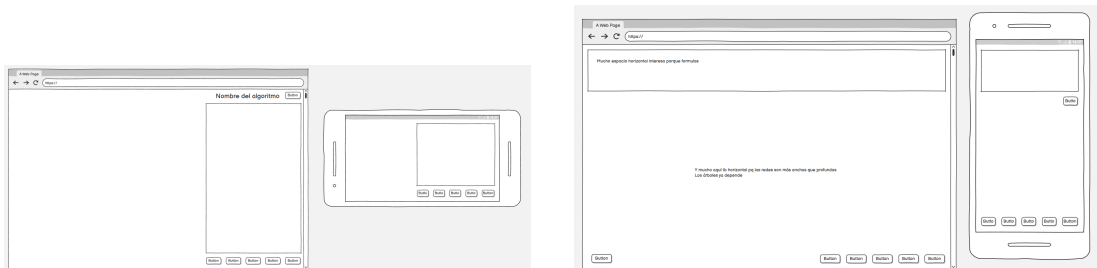


Figura 6.21: Mockups de las disposiciones horizontal y vertical de las vistas de algoritmo realizados con Balsamiq.

Bajo esta premisa el menú principal se reorganiza alrededor de contenedores de interfaz de Godot. Se incorporan márgenes, zonas centradas, paneles con anchura máxima y contenedores desplazables. Con esta estructura, el contenido puede reducir su anchura y ganar desplazamiento vertical cuando la pantalla es pequeña de forma automática. Los tamaños de fuente, los márgenes y las dimensiones mínimas de los botones se ajustan para conservar legibilidad sin depender de posiciones absolutas.

Tanto el árbol de decisión como la red neuronal se adaptan a esta lógica. Ahora la vista se divide en variantes vertical y horizontal, y el menú principal decide cuál cargar según la proporción de la ventana. La ventana deja de comportarse como un elemento flotante y queda integrada y fija en la escena. En pantallas anchas, la vista del algoritmo y la ventana informativa pueden situarse en paralelo, aprovechando mejor el espacio horizontal. En pantallas verticales, los elementos se colocan en una estructura apilada, con desplazamiento cuando el contenido excede el área visible. Esto facilita el uso en navegador y hace que los paneles principales respondan de forma más coherente al tamaño disponible.

Durante la creación de la red también se separa el espacio de edición del espacio de visualización. Antes, el panel de edición de la red estaba sobre la propia red mientras se editaba, como se aprecia en la Figura 6.22 igual tendría que poner más capturas. O quitar esta ref, claro. Ahora un contenedor agrupa el menú de creación y el panel de dataset, mientras que otro mantiene la red preliminar siempre visible como se ve en la Figura 6.23. Cuando comienza el entrenamiento o la inferencia, los paneles de creación se ocultan para dar prioridad a los controles del algoritmo y a la ventana informativa.

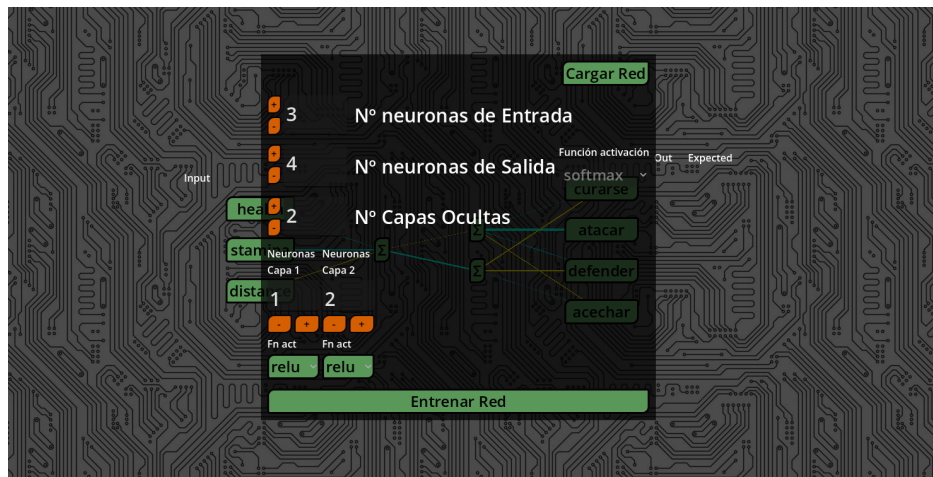


Figura 6.22: Creación de la red tras el menú traslúcido.

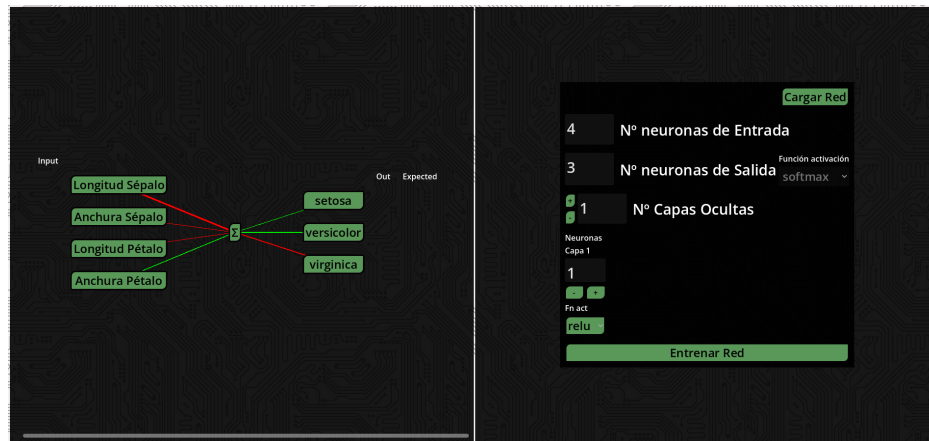


Figura 6.23: Creación de la red al lado del menú de creación.

A mayores, el redibujado de conexiones se optimiza en esta fase: al cambiar la arquitectura se eliminan solo las conexiones obsoletas, se actualizan las que siguen siendo válidas y se crean únicamente las nuevas. Esto hace que la previsualización sea más estable y clara cuando se modifica el número de capas o neuronas.

Para las zonas grandes, como el árbol entrenado o la red con varias capas, se mejora el desplazamiento y el zoom. El contenedor desplazable incorpora control con rueda, combinaciones de teclado y gestos táctiles. Con un dedo se puede arrastrar la vista, y con dos dedos se puede aplicar zoom mediante pinza. Estas interacciones son especialmente útiles en el árbol, donde algunas ramas pueden alejarse del centro, y en la red, donde muchas capas densas pueden ocupar bastante espacio.

El desplazamiento se revisa también para layouts con contenedores anidados. Si un panel interno llega a su límite de scroll, el evento puede propagarse a un contenedor superior. Así se evita que la rueda del ratón o el arrastre táctil queden atrapados en una zona que ya no puede desplazarse más. Y el zoom se implementa como un zoom global, que modifica el factor de escala de la raíz de la ventana dentro de unos límites definidos.

Los últimos cambios de cara a las pantallas consisten en reducir márgenes y separaciones entre los contenidos. En pantallas grandes, unos pocos píxeles añaden claridad visual, pero en pequeñas pueden ocupar una parte significativa del área útil. Por eso se recortan, no eliminan, márgenes y separaciones, conservando la organización de los paneles y ganando espacio para el contenido interactivo.

6.4.3 Identidad visual y legibilidad de la interfaz

También se revisa la estética general. El objetivo es que la aplicación tenga una identidad visual más consistente y que los elementos importantes destaquen con claridad. Se actualizan

colores de fondo, estilos de botones, bordes, textos y temas de paneles. El fondo pasa a usar una textura inspirada en una placa base.

También se mejoran los iconos y controles pequeños. El botón de menú deja de depender de un carácter textual y pasa a usar iconos gráficos. Esto evita diferencias entre fuentes o navegadores y ofrece un aspecto más estable en la exportación web.

En el árbol de decisión se ajusta el grosor de las conexiones y el contraste de las etiquetas de rama. El objetivo es transmitir un estilo parecido al de un árbol real, haciendo que las conexiones y nodos de decisión tengan colores cercanos al marrón. Las hojas se mantienen verdes, como hasta ahora.

En la red neuronal se revisa el color y grosor de las conexiones para representar mejor los pesos. Como se ve en la Figura 6.24 las conexiones hasta el momento se coloreaban con un gradiente en base al peso de la conexión. Las conexiones con peso positivo empezaban en un color cyan para valores bajos, que cambiaba a verde según el peso aumentaba. Mientras las conexiones con peso negativo empezaban en amarillo y cambiaban a rojo cuánto más negativo fuera el valor. Esta decisión se cambia para añadir más claridad: los pesos positivos se muestran siempre en verde, los negativos en rojo y los valores cercanos a cero en negro. La magnitud se sigue comunicando mediante el grosor, aunque se añaden límites mínimos y máximos a este para evitar líneas inapreciables o desproporcionadamente grandes.

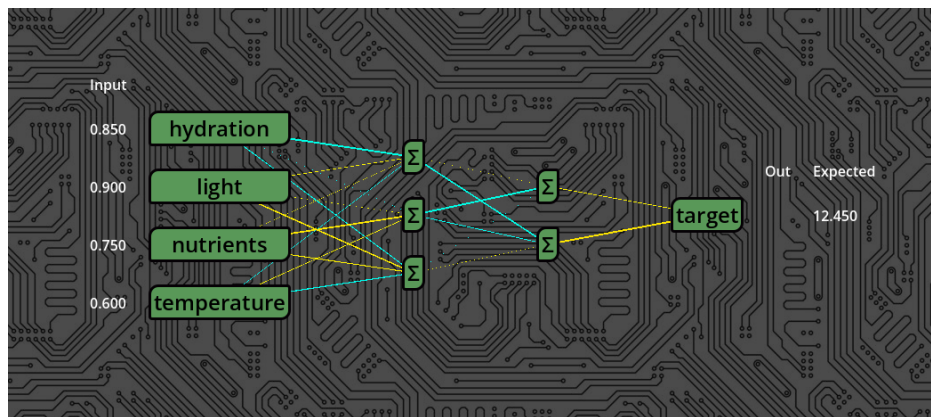


Figura 6.24: Imagen de la red en la que los colores de las conexiones son gradientes de avanzan desde el cyan hasta el verde para los valores positivos y del amarillo al rojo para los negativos.

Por último, durante el backpropagation de la red se añade un indicador visual más explícito para las actualizaciones de conexiones. Estas ya estaban animadas, pero si la magnitud del cambio no era muy grande, no era apreciable en muchos casos. El cambio consistió en añadir un círculo, cuyo color indica el signo de la actualización de forma intuitiva, que retrocede por la conexión que está siendo actualizada. Esto clarificaba que siempre se actualizan las conexiones.

6.4.4 Renderizado de fórmulas matemáticas

La ventana informativa de la red y la del árbol necesitan mostrar expresiones matemáticas para completar las explicaciones. Al principio, las fórmulas podían representarse como texto plano, pero esa solución era limitada para explicar operaciones como la derivada de los pesos. Por ello se dedica una tarea específica a integrar un sistema de renderizado LaTeX dentro de la aplicación.

La primera decisión consiste en tratar la fórmula como un nuevo tipo de Nodo personalizado que hereda del tipo Control. Al pertenecer esta clase, se puede colocar dentro de paneles, zonas desplazables y estructuras verticales junto al texto explicativo. Esto evita tener que posicionar manualmente las fórmulas como si fueran sprites de la Escena.

Inicialmente se prueba una solución basada en MathJax y elementos HTML superpuestos al canvas de Godot. Esa alternativa permite aprovechar el navegador, aunque la complejidad de mantener dos sistemas visuales coordinados hace preferible una solución integrada en Godot.

La solución final se desarrolla como una extensión del motor propia programada en Rust. El renderizador convierte expresiones LaTeX en SVG usando la librería Ratex [16] y las presenta como texturas dentro de Godot. El componente mantiene caché de fórmulas según texto, color, escala, tamaño de fuente y padding, y emite señales cuando el renderizado se completa o falla. También se prepara su compilación para web mediante WebAssembly, de manera que las fórmulas funcionen en la versión exportada a HTML5.

Una vez integrado, el renderizador se incorpora a la ventana de la red neuronal y a la del árbol de decisión. En la red se usa para acompañar las explicaciones del forward pass, la pérdida, los deltas y la actualización de pesos. En el árbol se usa para mostrar la fórmula de la entropía en los pasos donde el algoritmo compara atributos. Cuando la ventana muestra otros contenidos, como selección de nodos o evaluación de datos, la fórmula se oculta para no mantener información fuera de contexto.

otro sitio donde creo que voy a poner un pretérito El ajuste de escalado requirió de varias iteraciones. Se revisaron tamaños de fuente, padding, modo de estirado, tamaño mínimo lógico y normalización de SVG para que la fórmula tenga una dimensión natural dentro del panel. Este trabajo es especialmente importante en web, donde un escalado incorrecto puede producir texturas borrosas o demasiado grandes.

6.4.5 Selección de datasets y validaciones de inferencia

La carga de datos ya existía en fases anteriores, aunque en esta etapa se revisa para hacerla más flexible y segura. El árbol y la red tienen necesidades distintas, pero ambos se benefician de un panel de selección más claro y de validaciones antes de entrenar o evaluar.

En el árbol de decisión se amplían los datasets internos. El conjunto de frutas incorpora más instancias, clases con características compartidas y casos que pueden generar hojas impuras. También se añade el conjunto de tenis, un ejemplo clásico de árboles de decisión con atributos categóricos. Estos datasets permiten observar estructuras más variadas que las usadas en las primeras pruebas. Y a mayores se incorpora una característica que ya estaba en el selector de datasets de la red, pero no en el del árbol: el poder seleccionar la variable objetivo.

En la red neuronal se revisan también los conjuntos internos. El dataset de prueba inicial se sustituye por Iris, un ejemplo clásico de clasificación multiclase con cuatro entradas numéricas y salida *softmax*.

También se añaden validaciones básicas de estructura. La aplicación informa si el archivo no puede abrirse, si no contiene suficientes columnas, si no hay filas válidas o si todavía no se ha escogido una variable objetivo. El botón de selección permanece desactivado hasta que la configuración es suficiente para continuar. Así se evita iniciar el algoritmo con datos incompletos.

Por otra parte, la carga de datasets para inferencia requiere validaciones propias. Cuando el usuario evalúa un árbol o una red entrenada, el nuevo dataset debe ser compatible con el usado durante el entrenamiento. En el árbol se comprueba que estén los atributos requeridos y la variable objetivo esperada. En la red se comprueba que las entradas y salidas tengan la misma estructura que el modelo aprendió. Estas comprobaciones impiden evaluar con columnas ausentes, extra o en un orden incompatible.

Por último se traducen todos los datasets al castellano. Los nombres de atributos y valores pasan a estar alineados con el idioma de la interfaz y de la memoria. Esto evita mezclar explicaciones en castellano con ejemplos en inglés, y hace que los textos generados por la ventana resulten más naturales.

6.4.6 Evaluación visual del árbol de decisión

La evaluación del árbol se convierte en una de las tareas más importantes de esta fase. Ya existía una primera versión en la que los datos recorrían el árbol hasta una hoja, aunque todavía faltaba pulir la representación y la explicación de cada decisión.

Se comienza mejorando la representación visual de cada dato. Los datos dejan de ser cuadrados planos, y pasan ser botones, Nodos como los de la interfaz, pero con una estética característica. Cuando son creados, cada uno recibe un número único en base a la etiqueta que representan mediante una función hash. Con este valor, se establecen características visuales como el color o la forma. De esta forma, todos los datos a evaluar que tienen la misma etiqueta tienen el mismo aspecto. A esta característica se le añadió un extra basado en que si una de las características de los datos era el color, como en el caso del dataset de frutas, el color establecido para el dato a evaluar sería similar al que establece su atributo.

Cuando varios datos llegan a la misma hoja, se apilan debajo de ella. Esta solución evita superposiciones y permite ver cuántos ejemplos han terminado en una clasificación concreta. Si la pila queda fuera del área inicial del árbol, el canvas se amplía para que siga siendo accesible mediante scroll.

Otra mejora introducida es que el avance pasa a producirse nodo a nodo. En lugar de mover un dato desde la raíz hasta la hoja en una única animación, la aplicación mantiene un dato activo y un nodo activo. Cada pulsación avanza una decisión: primero el dato entra por la raíz, después se escoge la rama correspondiente al valor del atributo, y finalmente llega a una hoja. Este cambio hace que la evaluación siga la misma filosofía didáctica que permite que el usuario pueda observar una decisión concreta antes de pasar a la siguiente.

Para hacer robustas las animaciones, cada dato mantiene su propia cola de movimientos. Así, si se encadenan desplazamientos o se pulsa de nuevo mientras una animación está en curso, el flujo no depende de un único estado global. Cada elemento visual sabe qué movimientos tiene pendientes y los ejecuta en orden.

Mientras, la ventana informativa acompaña el recorrido. Al seleccionar un dato, muestra su etiqueta real y sus atributos. Al avanzar por el árbol, explica si el dato acaba de entrar por la raíz, si está pasando por un nodo interno o si ha llegado a una hoja. En la hoja compara la etiqueta real con la clase asignada por el árbol. Si coinciden, indica que la clasificación ha sido correcta. Si no coinciden, distingue entre una hoja pura y una hoja impura: en el primer caso, el entrenamiento no contenía ejemplos de esa etiqueta en esa región; en el segundo, la etiqueta real no fue la mayoritaria entre los ejemplos que llegaron a esa hoja. Y si se pulsa uno de los datos que se está evaluando la ventana muestra su información interna: los atributos y etiqueta del dato. Esto permite inspeccionar el árbol ya construido y entender qué información estadística hay detrás de cada parte de la estructura. Todo al mismo tiempo que se puede seguir haciendo click sobre los nodos del árbol para obtener información sobre ellos.

Se puede ver este proceso en la Figura 6.25.

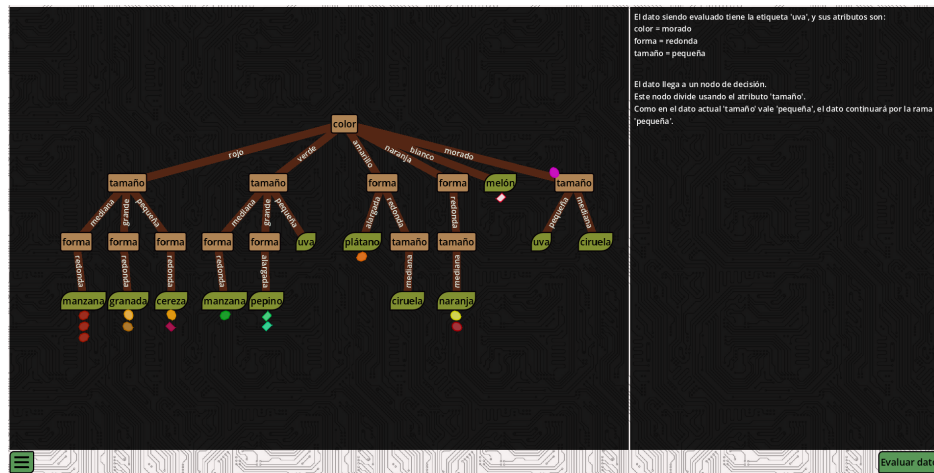


Figura 6.25: Árbol de decisión en medio del proceso de evaluación del dataset de frutas.

6.4.7 Gráficas de activación e inspección de neuronas y nodos

Se incorpora en la red neuronal una interacción sobre las neuronas y los nodos del árbol. Ahora el usuario puede pulsarlos para cambiar el texto en la ventana explicativa, que estos se queden resaltados destacando que la información refiere a ellos, y volver a ser pulsados para volver a la información que estaba antes. También se puede continuar con el avance del algoritmo para seguir con la explicación, o inspeccionar otros nodos o neuronas, algo importante de cara a que el usuario sienta que la aplicación no interfiere con lo que quiere hacer.

Y la explicación de las funciones de activación se refuerza con gráficas. Se generan imágenes para funciones como ReLU, sigmoide e identidad. Las gráficas se preparan con fondo transparente, ejes claros y cuadrícula tenue para integrarse en la estética oscura de la aplicación. Después se incorporan como texturas a la ventana informativa de la red. Durante el forward pass, la ventana puede mostrar la gráfica asociada a la función de activación de la capa que se está calculando. Cuando la explicación pasa a otro tipo de contenido, como la carga de datos, la retropropagación o el fin del entrenamiento, la gráfica se limpia junto con la fórmula anterior. Esto mantiene la ventana centrada en el paso actual y evita que una representación matemática permanezca visible fuera de contexto.

El último paso consiste en marcar sobre la gráfica el punto evaluado por la neurona. La ventana toma la entrada ponderada, la salida activada y la función correspondiente, y transforma esos valores en una posición dentro del área útil de la imagen. Sobre ella se dibuja un marcador rojo que indica qué punto de la curva se está usando en ese cálculo concreto. Así se conectan tres niveles de explicación: el valor numérico de entrada, la fórmula aplicada y la forma de la función de activación, todo dentro de la ventana explicativa.

6.4.8 Persistencia, carga y validación de modelos

La última tarea transversal se centra en conservar resultados, recuperarlos desde archivo y aumentar la confianza en los cálculos que muestra la aplicación. Esta necesidad aparece en los dos modelos y se resuelve con formatos distintos. En el árbol de decisión interesa guardar una estructura visual y lógica propia de la aplicación. En la red neuronal interesa guardar los parámetros de un modelo entrenado en un formato que pueda usarse también fuera de Godot.

En la versión web, tanto la carga como el guardado de archivos necesitan una integración específica con el navegador. Para ello se utiliza un puente de JavaScript que permite abrir selectores HTML, leer archivos elegidos por el usuario y generar descargas desde la aplicación exportada. Esta capa tiene un papel transversal: se usa para los modelos del árbol, para los modelos de la red y para cualquier operación de persistencia que deba atravesar las restricciones del navegador. Después de esta comunicación con JavaScript, Godot puede tratar el archivo cargado o preparar el contenido que se descargará.

En el árbol de decisión se implementa guardado y carga en formato JSON. El guardado conserva la estructura del árbol, la información visible de sus nodos y ramas, y los metadatos del dataset cuando están disponibles. Esto permite recuperar un árbol con su forma visual y con la información necesaria para seguir inspeccionándolo o evaluándolo.

La carga reconstruye el árbol a partir del archivo guardado. Primero valida la estructura, limpia el árbol actual y vuelve a crear los nodos. Después restaura la raíz, reorganiza la vista y actualiza el tamaño del canvas. Tras una carga válida, el controlador configura el árbol como una estructura ya entrenada. Para ello oculta los controles de entrenamiento, muestra la opción de evaluación y prepara el estado interno para que el árbol cargado pueda clasificarse directamente. Si el archivo no contiene metadatos de atributos, la aplicación intenta inferirlos a partir de la raíz o de los atributos usados por los nodos. Esta recuperación hace que la carga sea más útil incluso con archivos generados en fases anteriores.

En la red neuronal, la carga de modelos se sitúa en el menú de creación, ya que es ahí donde se define o se sustituye la arquitectura antes de entrenar o evaluar. La aplicación abre un cuadro para escoger el archivo y trabaja con ONNX [17] como formato implementado para esta operación. ONNX, siglas de *Open Neural Network Exchange*, es un formato abierto para representar modelos de aprendizaje automático y facilitar su intercambio entre herramientas compatibles. Los modelos importados se procesan en un Nodo dedicado, que convierte pesos, sesgos y funciones de activación a las estructuras internas de la aplicación. Si la carga falla, el menú muestra un mensaje de error; si se completa, la red visible se reconstruye con el modelo importado.

Antes de confirmar que la red cargada puede usarse, la aplicación valida su compatibilidad con el dataset seleccionado. El modelo debe tener el número esperado de neuronas de entrada y de salida. Cuando alguna dimensión no coincide, la interfaz informa de la diferencia entre la

arquitectura cargada y la requerida por los datos actuales. Esta comprobación impide entrenar o ejecutar inferencia con un modelo que no puede recibir las columnas seleccionadas o que no produce las salidas esperadas.

El guardado de la red se realiza mediante un panel de exportación que permite generar una representación del modelo entrenado para usarla fuera de la aplicación. Esa representación está basada en el estándar ONNX. Y como los archivos de exportación son creados por la propia aplicación, sin herramientas externas consolidadas, se realizó una verificación con un script de Python para asegurar que el proceso de transformar una red a ONNX se realizaba correctamente. Para ello se revisan todos los parámetros de la red y se comprueba que los valores corresponden con los que se ven en la aplicación, como se ve en la Figura 6.1. Estas pruebas confirman que la exportación conserva los parámetros escritos en ONNX y que una red externa compatible puede reconstruirse dentro de la aplicación.

Con todas estas mejoras, la aplicación queda más cohesionada. El menú principal presenta mejor los algoritmos, las vistas se adaptan a distintos dispositivos, las fórmulas se renderizan dentro de Godot también en web, los datasets se seleccionan con más control, la evaluación del árbol se vuelve paso a paso, la red ofrece explicaciones a varias escalas, las funciones de activación se acompañan con gráficas y los modelos pueden cargarse, guardarse o contrastarse con herramientas externas. El proyecto termina así con una interfaz más entendible y con una correspondencia más sólida entre los cálculos internos, su representación visual y su explicación textual.

Conclusiones

En este capítulo se recoge la situación final del trabajo y se valoran los resultados obtenidos. Tras el desarrollo descrito en el capítulo anterior, la aplicación ya permite construir y evaluar un árbol de decisión, entrenar una red neuronal paso a paso, inspeccionar sus cálculos internos y conservar resultados para contrastarlos fuera del propio entorno de Godot.

7.1 Resultados obtenidos y validación

Una parte importante del resultado final consiste en comprobar que lo mostrado por la aplicación se corresponde con modelos reales y con cálculos compatibles con herramientas externas. Esta validación se realizó en dos direcciones: por un lado, comparando la estructura del árbol de decisión generado dentro de la aplicación con una referencia obtenida fuera de Godot; por otro, comprobando que los pesos de una red neuronal pueden exportarse e importarse mediante ONNX sin perder su significado.

En el caso del árbol de decisión, la comprobación se hizo con un dataset categórico sintético. La Figura 7.1 muestra la estructura obtenida desde un script externo, impresa en la terminal. La captura recoge solo los primeros nodos del árbol completo por limitación de espacio. Esa parte permite identificar la raíz, las primeras ramas y varias hojas resultantes.

Al entrenar el árbol con ese mismo conjunto de datos desde la aplicación se obtiene la estructura de la Figura 7.2. La coincidencia entre ambas salidas se aprecia en la raíz, en las ramas principales y en las etiquetas finales asociadas a las hojas. Esta comparación confirma que la implementación del algoritmo en Godot toma las mismas decisiones de división que la referencia externa para este caso de prueba, y que la representación visual mantiene la información lógica necesaria para interpretar el modelo entrenado.

La red neuronal se validó desde una perspectiva diferente. En este caso, además de comprobar que el entrenamiento actualiza los parámetros dentro de la aplicación, se revisó que esos parámetros puedan leerse fuera de ella. La Figura 7.3 muestra el estado de una red en-

```

[Nodo 0] color (pureza: 4/22 = 0.1818)
|-- amarillo
| [Nodo 1] forma (pureza: 1/3 = 0.3333)
| |-- alargada
| | [Nodo 2] plátano (pureza: 1/1 = 1.0000)
| |-- redonda
| | [Nodo 3] tamaño (pureza: 1/2 = 0.5000)
| | |-- mediana
| | | [Nodo 4] ciruela (pureza: 1/2 = 0.5000)
|-- blanco
| [Nodo 5] melón (pureza: 1/1 = 1.0000)
|-- morado
| [Nodo 6] tamaño (pureza: 2/3 = 0.6667)
| |-- mediana
| | [Nodo 7] ciruela (pureza: 2/2 = 1.0000)
| |-- pequeña
| | [Nodo 8] uva (pureza: 1/1 = 1.0000)
|-- naranja
| [Nodo 9] forma (pureza: 1/2 = 0.5000)
| |-- redonda
| | [Nodo 10] tamaño (pureza: 1/2 = 0.5000)
| | |-- mediana
| | | [Nodo 11] naranja (pureza: 1/2 = 0.5000)
|-- rojo
| [Nodo 12] tamaño (pureza: 3/8 = 0.3750)
| |-- grande
| | [Nodo 13] forma (pureza: 1/2 = 0.5000)
| | |-- redonda
| | | [Nodo 14] granada (pureza: 1/2 = 0.5000)
| |-- mediana
| | [Nodo 15] forma (pureza: 3/4 = 0.7500)
| | |-- redonda
| | | [Nodo 16] manzana (pureza: 3/4 = 0.7500)
| |-- pequeña
| | [Nodo 17] forma (pureza: 1/2 = 0.5000)
| | |-- redonda
| | | [Nodo 18] cereza (pureza: 1/2 = 0.5000)
|-- verde
| [Nodo 19] tamaño (pureza: 1/5 = 0.2000)
| |-- grande
| | [Nodo 20] forma (pureza: 1/2 = 0.5000)
| | |-- alargada
| | | [Nodo 21] pepino (pureza: 1/2 = 0.5000)
| |-- mediana

```

Figura 7.1: Primeros nodos de la estructura final de un árbol de decisión obtenido con un dataset sintético desde un script externo.

trenada en la propia aplicación. En la neurona de salida se observan pesos cercanos a 1.943, 1.929 y -0.965 , junto con un sesgo cercano a 0.994.

Después de exportar esa red a ONNX, un script externo carga el archivo generado y muestra los parámetros de salida. Como se ve en la Figura 7.4, los valores leídos desde el archivo son prácticamente los mismos que los mostrados por la aplicación. Las pequeñas diferencias se deben al redondeo visual y a la precisión numérica usada en cada entorno. Con esto se comprueba que la exportación conserva el orden y el valor de los parámetros aprendidos.

La validación también se realizó en el sentido inverso. Para ello se generó fuera de la aplicación una red sintética con pesos exactos 2.0, 2.0 y -1.0 , y con sesgo 1.0. La salida de terminal de la Figura 7.5 recoge esos valores antes de importar el modelo en Godot.

Al importar ese archivo en la aplicación, la red reconstruida conserva los mismos parámetros, tal como se muestra en la Figura 7.6. Esta prueba completa la comprobación de interoperabilidad: los modelos entrenados en la aplicación pueden salir hacia ONNX, y los modelos creados fuera pueden entrar en la aplicación con sus pesos y sesgos correctamente asignados.

Estas pruebas refuerzan la validez del resultado final. En el árbol de decisión, la estructura visual coincide con una construcción externa equivalente. En la red neuronal, los parámetros que se ven durante el entrenamiento son los mismos que se escriben en el archivo exportado y los mismos que se recuperan al importar un modelo conocido. Por tanto, la interfaz funciona como una animación didáctica y representa el estado real de los modelos que calcula, entrena y evalúa.

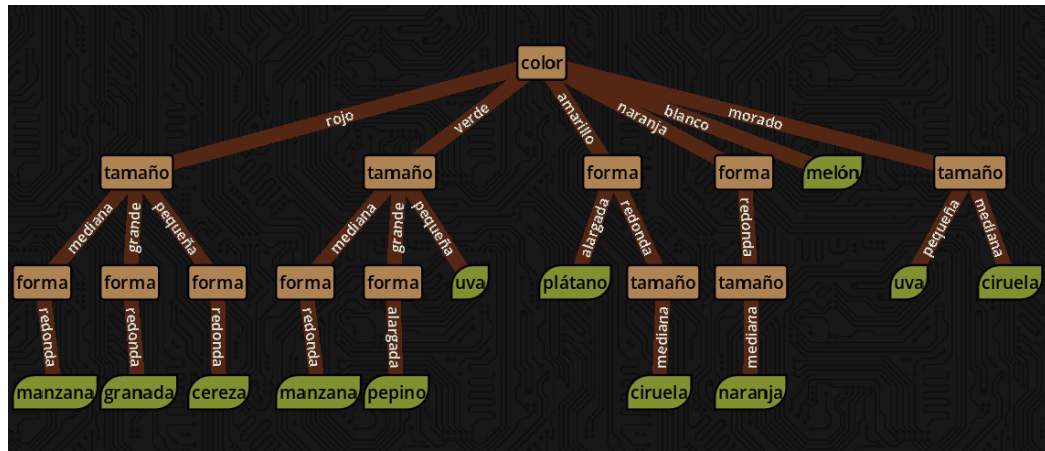


Figura 7.2: Árbol de decisión generado por la aplicación con el mismo dataset sintético usado en la referencia externa.

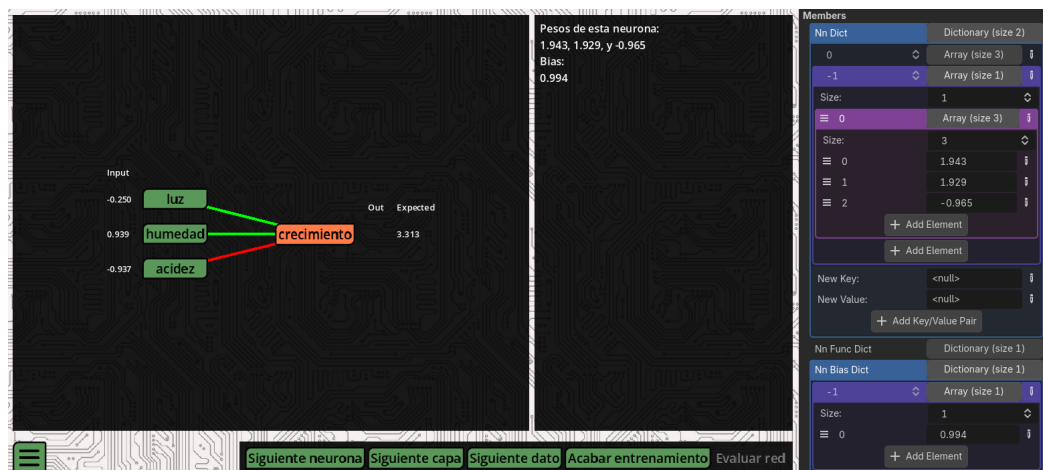


Figura 7.3: Red neuronal entrenada dentro de la aplicación, con los pesos y el sesgo de la neurona de salida visibles en la interfaz.

```

Éxito: El archivo 'network.onnx' se ha importado y verificado correctamente.

W_out:
[[ 1.9381292  1.9237686 -0.96262616]]

B_out:
[0.99369425]

```

Figura 7.4: Lectura externa del archivo ONNX exportado desde la aplicación, con los pesos y el sesgo de la red entrenada.

```
Pesos finales del modelo:  
tensor([[ 2.0000,  2.0000, -1.0000]])  
  
Bias final del modelo:  
tensor([1.0000])
```

Figura 7.5: Red sintética exportada a ONNX desde un entorno externo, con pesos y sesgo conocidos.

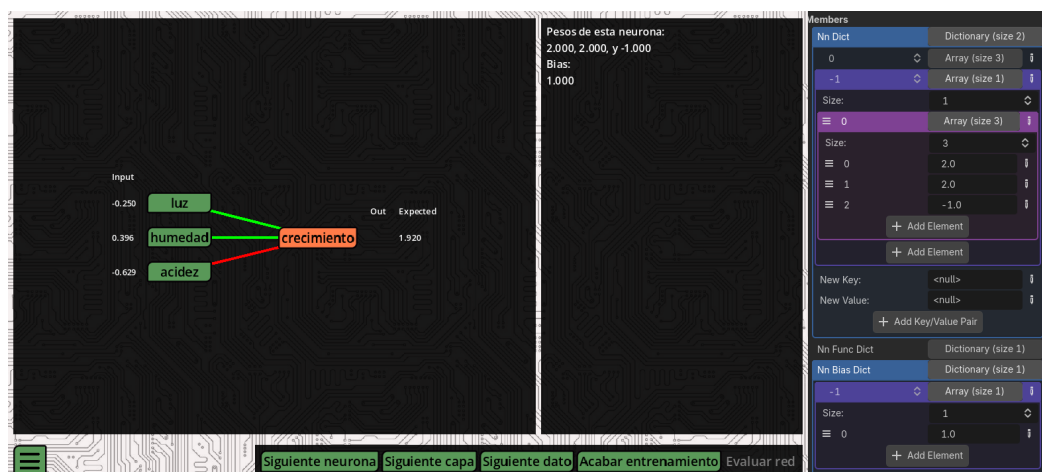


Figura 7.6: Red sintética importada en la aplicación desde ONNX, con los pesos y el sesgo esperados visibles en la interfaz.

Trabajo futuro

El desarrollo realizado deja una base sobre la que pueden plantearse varias ampliaciones. La aplicación actual se centra en dos modelos concretos, árboles de decisión y redes neuronales multicapa, y los presenta de forma incremental para que el usuario pueda seguir tanto el entrenamiento como la evaluación. A partir de esa base, el trabajo futuro puede organizarse en tres líneas principales: ampliar el catálogo de algoritmos, profundizar en las opciones de los modelos ya implementados e integrar el aprendizaje con simulaciones interactivas creadas en el propio motor.

8.1 Ampliación del catálogo de modelos

Una primera línea de evolución consiste en incorporar nuevos algoritmos de aprendizaje automático. La estructura modular desarrollada para separar algoritmo, representación visual y ventana informativa facilita que la aplicación pueda crecer con nuevos modelos, siempre que cada uno se acompañe de una visualización adaptada a su forma de aprender.

En el caso de las redes neuronales, una extensión natural sería incluir topologías más variadas. La versión actual trabaja con redes densas multicapa, suficientes para explicar el forward pass, el cálculo del error y la retropropagación. A partir de ahí, podrían añadirse arquitecturas convolucionales, recurrentes o basadas en mecanismos de atención. Cada una de ellas permitiría trabajar conceptos nuevos: filtros y mapas de características en redes convolucionales, dependencias temporales en redes recurrentes o matrices de atención en modelos de tipo transformer.

Los transformadores tendrían un interés especial como línea futura por su presencia en muchos sistemas actuales de aprendizaje automático. Su incorporación requeriría representar elementos distintos a los de una red densa clásica, como tokens, embeddings, cabezas de atención y bloques sucesivos de procesamiento. Esta ampliación exigiría una visualización más compleja, con relaciones simultáneas entre muchos elementos y conexiones que van más

allá de las capas consecutivas de una red densa.

La incorporación de nuevos modelos también permitiría comparar enfoques. La aplicación podría mostrar cómo distintos algoritmos resuelven un mismo conjunto de datos, qué información utiliza cada uno y qué tipo de explicación resulta más adecuada en cada caso. Esta comparación reforzaría el objetivo didáctico del proyecto, ya que ayudaría a entender que el aprendizaje automático agrupa técnicas con estructuras y procesos de decisión muy diferentes.

8.2 Profundización en los modelos existentes

Una segunda línea de trabajo consiste en enriquecer los algoritmos que ya forman parte de la aplicación. En el árbol de decisión, una mejora directa sería permitir distintos criterios de división. La versión actual se apoya en una métrica basada en entropía e información, por lo que el usuario sigue siempre el mismo tipo de razonamiento al crear cada nodo. Incluir alternativas como el índice Gini o el ratio de ganancia permitiría comparar cómo cambia la elección de atributos según la medida utilizada.

También sería relevante ampliar el tipo de datos aceptados por el árbol. En su estado actual, el algoritmo trabaja con atributos categóricos. Para manejar atributos numéricos continuos habría que buscar umbrales de corte y explicar por qué un valor separa mejor los datos que otros posibles valores. Esta mejora enriquecería la visualización, ya que la ventana informativa podría mostrar la comparación entre atributos y, dentro de cada atributo numérico, la comparación entre posibles puntos de división.

Otra ampliación posible para el árbol sería incorporar técnicas de poda o mecanismos de control de crecimiento. Con ello, el usuario podría observar la diferencia entre construir un árbol que se ajusta mucho a los datos de entrenamiento y construir una estructura más sencilla con mejor capacidad de generalización. Esta línea conectaría la visualización del algoritmo con problemas habituales del aprendizaje automático, como el sobreajuste y la validación del modelo.

En la red neuronal, el trabajo futuro puede avanzar hacia una configuración más flexible del entrenamiento. Algunas opciones posibles son permitir distintos optimizadores, definir el número de épocas, ajustar el tamaño de batch, modificar la tasa de aprendizaje durante la ejecución, elegir funciones de pérdida alternativas o incorporar regularización. Cada una de estas decisiones afecta al modo en el que cambian los pesos, por lo que la aplicación podría mostrar sus efectos de forma visual durante el backward pass.

La red también podría dejar de limitarse a conexiones densas. Permitir topologías no densas o definidas por el usuario abriría la puerta a explicar cómo cambia el flujo de información cuando una neurona no está conectada con todas las neuronas de la capa siguiente. Esta me-

jora mantendría la relación entre arquitectura visible y modelo lógico, a la vez que daría más importancia a la estructura de conexiones como parte del aprendizaje.

Todas estas ampliaciones comparten una misma condición: las nuevas opciones deberían integrarse sin perder el carácter explicativo de la herramienta. Añadir parámetros o variantes del algoritmo tendría sentido si la aplicación muestra qué cambia en el cálculo, qué decisión se toma y qué consecuencia tiene sobre el modelo final.

8.3 Integración con simulaciones interactivas

La tercera línea de trabajo es la más vinculada con la elección de Godot como herramienta de desarrollo. Una evolución especialmente interesante sería conectar los algoritmos con una simulación o un videojuego ejecutándose en la propia aplicación. En ese escenario, el entorno interactivo generaría datos durante la ejecución, y el modelo aprendería a partir de lo que sucede en la escena.

Esta integración permitiría pasar de datasets estáticos a datos producidos por una dinámica viva. Por ejemplo, una escena podría generar posiciones, acciones, colisiones, decisiones de un personaje o resultados obtenidos tras cada intento. Esos eventos podrían convertirse en entradas para un modelo, y la respuesta del modelo podría influir de nuevo en el comportamiento de la simulación. El usuario observaría entonces dos procesos conectados: la evolución del entorno y la evolución del algoritmo que aprende a partir de él.

Una línea de este tipo serviría también para explorar problemas cercanos al aprendizaje por refuerzo. Un agente podría recibir información del estado del juego, escoger acciones y obtener recompensas según lo ocurrido. La visualización tendría que mostrar tanto la ejecución del agente como la actualización interna del modelo. Esto ampliaría el proyecto hacia una dimensión más experimental, donde la explicación del aprendizaje estaría ligada a consecuencias visibles dentro de una escena interactiva.

Esta posibilidad refuerza la justificación del uso de Godot. El motor actuaría como soporte para dibujar interfaces, entorno de simulación, generador de datos y medio de interacción. Además, la exportación a HTML5 permitiría mantener el acceso desde navegador incluso en versiones más cercanas a una experiencia de juego educativo.

En conjunto, estas tres líneas de trabajo permitirían hacer crecer la aplicación en amplitud, profundidad y conexión con entornos interactivos. La prioridad debería seguir siendo la misma que en la versión actual: que cada avance del algoritmo pueda observarse, que cada decisión esté acompañada por una explicación y que la visualización ayude a entender el funcionamiento interno del modelo.

A mayores, sería de gran interés probar la aplicación con usuarios reales para detectar qué partes resultan más y menos claras, y qué cambios podrían mejorar su utilidad didáctica. Sería

especialmente interesante contar con personas en distintos estados del aprendizaje, desde usuarios que se acercan por primera vez al aprendizaje automático hasta estudiantes con las bases asentadas, así como con profesores que pudieran valorar su uso como material de apoyo en clases o actividades prácticas.

Apéndice

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 16 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 16 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>

- [11] GitHub, Inc., “Github pages documentation,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>
- [12] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [13] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [14] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [15] Balsamiq Studios, LLC, “Balsamiq wireframes,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://balsamiq.com/wireframes/>
- [16] ratex contributors, “ratex,” crates.io, consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://crates.io/crates/ratex>
- [17] ONNX contributors, “Open Neural Network Exchange,” consultado el 16 de junio de 2026. [En línea]. Disponible en: <https://onnx.ai/>